

Conjuntos de instruções: características e funções

10.1 Características das instruções de máquina

- Elementos de uma instrução de máquina
- Representação da instrução
- Tipos de instrução
- Número de endereços
- Projeto do conjunto de instruções

10.2 Tipos de operandos

- Números
- Caracteres
- Dados lógicos

10.3 Tipos de dados do Intel x86 e do ARM

- Tipos de dados do x86
- Tipos de dados do ARM

10.4 Tipos de operações

- Transferência de dados
- Aritméticas
- Lógicas
- Conversão
- Entrada/saída
- Controle do sistema
- Transferência de controle

10.5 Tipos de operação Intel x86 e do ARM

- Tipos de operação do x86
- Tipos de operação do ARM

10.6 Leitura recomendada

Apêndice 10A Pilhas

Apêndice 10B *Little, Big e Bi-endian*

PRINCIPAIS PONTOS

- Os elementos essenciais de uma instrução de computador são o *opcode* (código de operação), que especifica a operação a ser realizada; as referências de operando de origem e destino, que especificam os locais de entrada e saída para a operação; e a referência da próxima instrução, que normalmente é implícita.
- Os *opcodes* especificam operações em uma das seguintes categorias gerais: operações aritméticas e lógicas; movimentação de dados entre dois registradores, registrador e memória, ou dois locais de memória; E/S; e controle.
- As referências de operando especificam um local de registrador ou memória dos dados do operando. Os tipos dos dados podem ser endereços, números, caracteres ou dados lógicos.
- Um recurso arquitetural comum nos processadores é o uso da pilha, que pode ou não estar visível ao programador. As pilhas são usadas para gerenciar chamadas e retornos de procedimento, e podem ser fornecidas como uma forma alternativa de endereçar a memória. As operações básicas da pilha são PUSH, POP e operações sobre um ou dois locais no topo da pilha. As pilhas normalmente são implementadas para crescer de endereços maiores para endereços menores.
- Os processadores endereçáveis por byte podem ser categorizados como *big-endian*, *little-endian* ou *bi-endian*. Um valor numérico com múltiplos bytes, armazenado com o byte mais significativo no endereço numérico mais baixo, é armazenado no padrão *big-endian*. O estilo *little-endian* armazena o byte mais significativo no endereço numérico mais alto. Um processador *bi-endian* pode trabalhar com os dois estilos.

Grande parte do que discutimos neste livro não é prontamente aparente ao usuário ou programador. Se um programador estiver usando uma linguagem de alto nível, como Pascal ou Ada, muito pouco da arquitetura da máquina básica é visível.

Um limite onde o projetista de computador e o programador de computador podem ver a mesma máquina é o conjunto de instruções de máquina. Do ponto de vista do projetista, o conjunto de instruções de máquina oferece os requisitos funcionais para o processador: implementar o processador é uma tarefa que em grande parte envolve imple-

mentar o conjunto de instruções de máquina. O usuário que escolhe programar em linguagem de máquina (na realidade, em linguagem de montagem; veja Apêndice B) fica ciente da estrutura do registrador e da memória, dos tipos de dados aceitos diretamente pela máquina e do funcionamento da ALU.

Uma descrição do conjunto de instruções de máquina de um computador explica bastante sobre o processador. Consequentemente, focalizamos instruções de máquina neste capítulo e no seguinte.



10.1 Características das instruções de máquina

A operação do processador é determinada pelas instruções que ele executa, conhecidas como *instruções de máquina* ou *instruções do computador*. A coleção de diferentes instruções que o processador pode executar é conhecida como *conjunto de instruções* do processador.



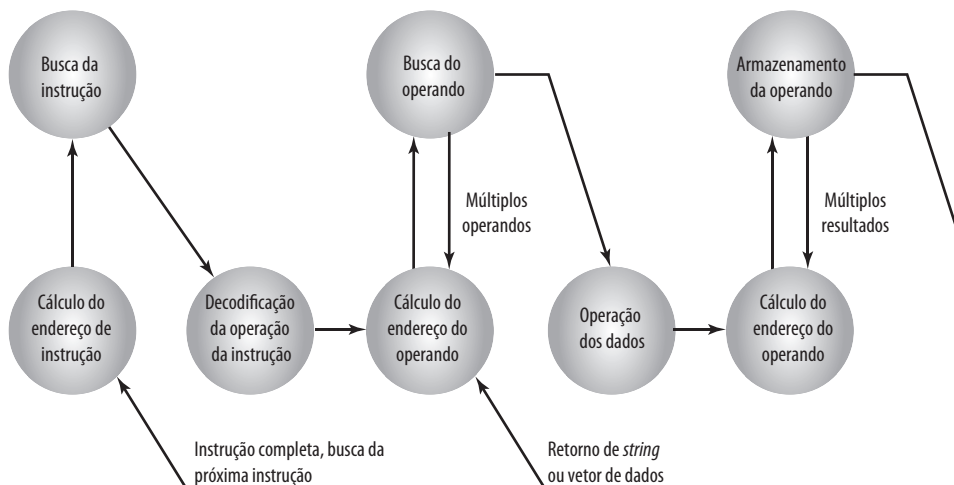
Elementos de uma instrução de máquina

Cada instrução precisa conter as informações exigidas pelo processador para execução. A Figura 10.1, que repete a Figura 3.6, mostra as etapas envolvidas na execução da instrução e, por implicação, define os elementos de uma instrução de máquina. Esses elementos são os seguintes:

- **Código de operação:** especifica a operação a ser realizada (por exemplo, ADD, E/S). A operação é especificada por um código binário, conhecido como código de operação, ou **opcode** (*operation code*).
- **Referência à operando fonte:** a operação pode envolver um ou mais operandos fontes, ou seja, operandos que são entradas para a operação.
- **Referência à operando destino:** a operação pode produzir um resultado.
- **Referência à próxima instrução:** isso diz ao processador onde buscar a próxima instrução após o término da execução desta instrução.

O endereço da próxima instrução a ser buscada poderia ser um endereço real ou um endereço virtual, dependendo da arquitetura. Geralmente, a distinção é transparente à arquitetura do conjunto de instruções. Na maior parte dos casos, a próxima instrução a ser buscada vem imediatamente após a instrução corrente. Nesses casos, não existe uma referência explícita à próxima instrução. Quando uma referência explícita é necessária, então o endereço da memória principal ou da memória virtual precisa ser fornecido. A forma como esse endereço é fornecido é discutida no Capítulo 11.

Figura 10.1 Diagrama de estado do ciclo de instrução



Operandos fonte e destino podem estar em uma destas quatro áreas:

- **Memória principal ou virtual:** assim como as referências à próxima instrução, o endereço da memória principal ou virtual precisa ser fornecido.
- **Registrador do processador:** com raras exceções, um processador contém um ou mais registradores que podem ser referenciados por instruções de máquina. Se houver apenas um registrador, a referência a ele pode ser implícita. Se houver mais de um registrador, então cada registrador recebe um nome ou número exclusivo, e a instrução precisa conter o número do registrador desejado.
- **Imediato:** o valor do operando está contido em um campo na instrução sendo executada.
- **Dispositivo de E/S:** a instrução precisa especificar o módulo e o dispositivo de E/S para a operação. Se a E/S mapeada na memória for usada, esse é apenas outro endereço da memória principal ou virtual.



Representação da instrução

Dentro do computador, cada instrução é representada por uma sequência de bits. A instrução é dividida em campos, correspondentes aos elementos constituintes da instrução. Um exemplo simples de um formato de instrução aparece na Figura 10.2. Como outro exemplo, o formato de instrução do IAS aparece na Figura 2.2. Com a maioria dos conjuntos de instruções, mais de um formato é utilizado. Durante a execução da instrução, uma instrução é lida para um registrador de instrução (IR) no processador. O processador precisa ser capaz de extrair os dados dos diversos campos da instrução para realizar a operação exigida.

É difícil tanto para o programador quanto para o leitor de livros-texto lidar com representações binárias das instruções de máquina. Assim, tornou-se uma prática comum usar uma *representação simbólica* das instruções de máquina. Um exemplo disso foi usado para o conjunto de instruções do IAS, na Tabela 2.1.

Os *opcodes* são representados por abreviações, chamadas *mnemônicos*, que indicam a operação. Alguns exemplos comuns são

ADD	Adiciona
SUB	Subtrai
MUL	Multiplica
DIV	Divide
LOAD	Carrega dados da memória
STOR	Armazena dados na memória

Operandos também são representados simbolicamente. Por exemplo, a instrução

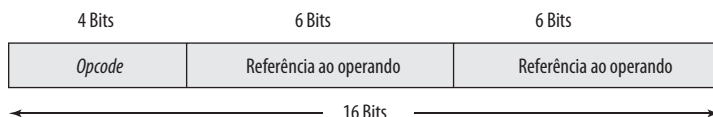
ADD R, Y

pode significar somar o valor contido no local de dados Y com o conteúdo do registrador R. Neste exemplo, Y refere-se ao endereço de um local na memória, e R refere-se a um registrador em particular. Observe que a operação é realizada sobre o conteúdo de um local, não sobre seu endereço.

Assim, é possível escrever um programa em linguagem de máquina em forma simbólica. Cada *opcode* tem uma representação binária fixa e o programador especifica o local de cada operando simbólico. Por exemplo, o programador poderia começar com uma lista de definições:

X = 513
Y = 514

Figura 10.2 Um formato de instrução simples



e assim por diante. Um programa simples aceitaria essa entrada simbólica, converteria os *opcodes* e as referências dos operandos para forma binária e construiria as instruções de máquina binárias.

Os programadores de linguagem de máquina são raros, quase inexistentes. A maioria dos programas hoje é escrita em uma linguagem de alto nível ou, fora isso, linguagem de montagem, que discutimos no Apêndice B. Porém, a linguagem de máquina simbólica continua sendo uma ferramenta útil para descrever instruções de máquina e vamos utilizá-la para essa finalidade.



Tipos de instrução

Considere uma instrução em linguagem de alto nível que poderia ser expressa em uma linguagem como BASIC ou FORTRAN. Por exemplo,

X = X + Y

Essa instrução orienta o computador a somar o valor armazenado em Y ao valor armazenado em X, colocando o resultado em X. Como isso poderia ser feito com instruções de máquina? Vamos supor que as variáveis X e Y correspondam aos locais 513 e 514. Se consideramos um conjunto simples de instruções de máquina, essa operação poderia ser feita com três instruções:

1. Carregue um registrador com o conteúdo do local de memória 513.
2. Some o conteúdo do local de memória 514 ao registrador.
3. Armazene o conteúdo do registrador no local de memória 513.

Como podemos ver, uma única instrução em BASIC pode exigir três instruções de máquina. Isso é típico do relacionamento entre uma linguagem de alto nível e uma linguagem de máquina. Uma linguagem de alto nível expressa operações em uma forma algébrica concisa, usando variáveis. Uma linguagem de máquina expressa operações em uma forma básica envolvendo a movimentação de dados de e para os registradores.

Com esse exemplo simples para nos guiar, vamos considerar os tipos de instruções que precisam ser incluídas em um computador prático. Um computador deve ter um conjunto de instruções que permita ao usuário formular qualquer tarefa de processamento de dados. Outro modo de ver isso é considerar as capacidades de uma linguagem de programação de alto nível. Qualquer programa escrito em uma linguagem de alto nível, para ser executado, precisa ser traduzido para linguagem de máquina. Assim, o conjunto de instruções de máquina precisa ser suficiente para expressar qualquer uma das instruções de uma linguagem de alto nível. Com isso em mente, podemos categorizar os tipos de instrução da seguinte forma:

- **Processamento de dados:** instruções aritméticas e lógicas.
- **Armazenamento de dados:** movimentação de dados para dentro ou fora do registrador e/ou locais de memória.
- **Movimentação de dados:** instruções de E/S.
- **Controle:** instruções de teste e desvio.

As instruções *aritméticas* oferecem capacidades de cálculo para o processamento de dados numéricos. As instruções *lógicas* (booleanas) operam sobre os bits de uma palavra como bits, e não como números; assim, elas oferecem capacidades de processamento de qualquer outro tipo de dado que o usuário possa querer empregar. Essas operações são realizadas principalmente sobre os dados nos registradores do processador. Portanto, deve haver instruções de *memória* para mover dados entre a memória e os registradores. As instruções de *E/S* são necessárias para transferir programas e dados para a memória e os resultados de cálculos de volta ao usuário. As instruções de *teste* são usadas para testar o valor de uma palavra de dados ou o *status* de um cálculo. As instruções de *desvio* são então usadas para desviar para um conjunto de instruções diferente, dependendo da decisão tomada.

Vamos examinar os diversos tipos de instruções com mais detalhes mais adiante neste capítulo.



Números de endereços

Uma das formas tradicionais de descrever a arquitetura do processador é em termos do número de endereços contidos em cada instrução. Essa dimensão tornou-se menos significativa com o aumento da complexidade de projeto do processador. Apesar disso, é útil neste ponto considerar e analisar essa distinção.

Qual é o número máximo de endereços que poderia ser preciso em uma instrução? Evidentemente, as instruções aritméticas e lógicas exigirão mais operandos. Praticamente todas as operações aritméticas e lógicas são

unárias (um operando de origem) ou binárias (dois operandos de origem). Assim, precisaríamos de um máximo de dois endereços para referenciar operandos de origem. O resultado de uma operação precisa ser armazenado, sugerindo um terceiro endereço, que define um operando de destino. Finalmente, após o término de uma instrução, a próxima instrução precisa ser buscada, e seu endereço é necessário.

Essa linha de raciocínio sugere que uma instrução poderia plausivelmente ter que conter quatro referências de endereço: dois operandos de origem, um operando de destino e o endereço da próxima instrução. Na maioria das arquiteturas, quase todas as instruções possuem um, dois ou três endereços de operando, com o endereço da próxima instrução sendo implícito (obtido pelo contador de programa – PC). A maioria das arquiteturas também possui algumas instruções de uso especial, com mais operandos. Por exemplo, as instruções de *load* e *store* múltiplo da arquitetura ARM, descritas no Capítulo 11, designam até 17 operandos de registrador em uma única instrução.

A Figura 10.3 compara instruções típicas de um, dois e três endereços, que poderiam ser usadas para calcular $Y = (A - B) / [C + (D \times E)]$. Com três endereços, cada instrução especifica dois locais de operandos de origem e um local de operando de destino. Como escolhemos não alterar o valor de qualquer um dos locais de operando, um local temporário, T, é usado para armazenar alguns resultados intermediários. Observe que existem quatro instruções e que a expressão original tinha cinco operandos.

Formatos de instrução de três endereços não são comuns, pois exigem um formato de instrução relativamente longo para manter as três referências de endereço. Com instruções de dois endereços, e para operações binárias, um endereço precisa realizar o trabalho duplo como um operando e como um resultado. Assim, a instrução SUB Y, B executa o cálculo $Y - B$ e armazena o resultado em Y. O formato de dois endereços reduz o requisito de espaço, mas também introduz algumas coisas estranhas. Para evitar alterar o valor de um operando, a instrução MOVE é usada para mover um dos valores para um resultado ou local temporário antes de realizar a operação. Nosso programa de exemplo se expande para seis instruções.

Mais simples ainda é a instrução de um endereço. Para que esta funcione, um segundo endereço precisa ser implícito. Isso era comum nas máquinas mais antigas, com o endereço implícito sendo um registrador do processador conhecido como **acumulador** (AC). O acumulador contém um dos operandos e é usado para armazenar o resultado. Em nosso exemplo, oito instruções são necessárias para realizar a tarefa.

Na verdade, é possível termos zero endereços para algumas instruções. Instruções de zero endereços se aplicam a uma organização de memória especial, chamada *pilha*. Uma pilha é um conjunto de locais do tipo *last-in-first-out* (último a entrar, primeiro a sair). A pilha está em um local conhecido e, frequentemente, pelo menos os dois ele-

Figura 10.3 Programas para executar $Y = \frac{A - B}{C + (D \times E)}$

Instrução	Comentário
SUB Y, A, B	$Y \leftarrow A - B$
MPY T, D, E	$T \leftarrow D \times E$
ADD T, T, C	$T \leftarrow T + C$
DIV Y, Y, T	$Y \leftarrow Y \div T$

(a) Instruções com três endereços

Instrução	Comentário
MOVE Y, A	$Y \leftarrow A$
SUB Y, B	$Y \leftarrow Y - B$
MOVE Y, D	$T \leftarrow D$
MPY T, E	$T \leftarrow T \times E$
ADD T, C	$T \leftarrow T + C$
DIV Y, T	$Y \leftarrow Y \div T$

(b) Instruções de dois endereços

Instrução	Comentário
LOAD D	$AC \leftarrow D$
MPY E	$AC \leftarrow AC \times E$
ADD C	$AC \leftarrow AC + C$
STOR Y	$Y \leftarrow AC$
LOAD A	$AC \leftarrow A$
SUB B	$AC \leftarrow AC - B$
DIV Y	$AC \leftarrow AC \div T$
STOR Y	$Y \leftarrow AC$

(c) Instruções de um endereço

mentos do topo estão nos registradores do processador. Assim, instruções com zero endereços referenciariam os dois elementos do topo da pilha. As pilhas são descritas no Apêndice 10A. Seu uso é explicado melhor mais adiante neste capítulo e no Capítulo 11.

A Tabela 10.1 resume as interpretações das instruções com zero, um, dois ou três endereços. Em cada caso na tabela, considera-se que o endereço da próxima instrução é implícito e que uma operação com dois operandos de origem e um operando de resultado deve ser realizada.

O número de endereços por instrução é uma decisão básica de projeto. Menos endereços por instrução resulta em instruções que são mais primitivas, exigindo um processador menos complexo. Isso também resulta em instruções de menor tamanho. Por outro lado, os programas contêm mais instruções no total, o que, em geral, resulta em tempos maiores de execução e programas maiores e mais complexos. Além disso, existe um limite importante entre instruções de um endereço e múltiplos endereços. Com instruções de um endereço, o programador geralmente tem à sua disposição apenas um registrador de uso geral, o acumulador. Com instruções de múltiplos endereços, é comum ter múltiplos registradores de uso geral. Isso permite que algumas operações sejam realizadas unicamente sobre registradores. Como as referências a registradores são mais rápidas que as referências à memória, tem-se rapidez na execução. Por motivos de flexibilidade e capacidade de usar múltiplos registradores, a maioria das máquinas contemporâneas emprega uma mistura de instruções de dois e três endereços.

As decisões de projeto envolvidas na escolha do número de endereços por instrução são complicadas por outros fatores. Existe a questão de se um endereço referencia um local da memória ou um registrador. Como existem menos registradores, menos bits são necessários para uma referência de registrador. Além disso, conforme veremos no próximo capítulo, uma máquina pode oferecer uma série de modos de endereçamento e a especificação do modo exige um ou mais bits. O resultado é que a maioria dos projetos de processador envolve uma série de formatos de instrução.



Projeto do conjunto de instruções

Um dos aspectos mais interessantes e mais analisados do projeto de computador é o projeto do conjunto de instruções. O projeto de um conjunto de instruções é muito complexo, pois afeta muitos aspectos do sistema de computador. Ele define muitas das funções realizadas pelo processador e, portanto, tem um efeito significativo sobre a implementação do processador. O conjunto de instruções é o meio de o programador controlar o processador. Assim, os requisitos do programador precisam ser considerados no projeto do conjunto de instruções.

Pode ser surpresa para você saber que algumas das questões mais fundamentais em relação ao projeto dos conjuntos de instruções continuam em discussão. Na realidade, nos últimos anos, o nível de divergência com relação a esses fundamentos realmente cresceu. As questões básicas mais importantes de projeto são as seguintes:

- **Repertório de operações:** quantas e quais operações oferecer, e que complexidade as operações deverão ter.
- **Tipos de dados:** os diversos tipos de dados sobre os quais as operações são realizadas.

Tabela 10.1 Utilização de endereços de instrução (instruções sem desvio)

Número de endereços	Representação simbólica	Interpretação
3	OP A, B, C	$A \leftarrow B \text{ OP } C$
2	OP A, B	$A \leftarrow A \text{ OP } B$
1	OP A	$AC \leftarrow AC \text{ OP } A$
0	OP	$T \leftarrow (T - 1) \text{ OP } T$

AC = acumulador

T = topo da pilha

(T - 1) = segundo elemento da pilha

A, B, C = locais de memória ou registradores

- **Formato de instrução:** tamanho da instrução (em bits), número de endereços, tamanho dos diversos campos, e assim por diante.
- **Registradores:** número de registradores do processador que podem ser referenciados pelas instruções e seu uso.
- **Endereçamento:** o modo ou modos pelos quais o endereço de um operando é especificado.

Essas questões são altamente inter-relacionadas e precisam ser consideradas juntas no projeto de um conjunto de instruções. Este livro, naturalmente, precisa considerá-las em alguma sequência, mas tentamos mostrar os inter-relacionamentos.

Devido à importância desse assunto, muito da Parte 3 é dedicado ao projeto do conjunto de instruções. Após esta seção introdutória, este capítulo examina os tipos de dados e o repertório de operações. O Capítulo 11 examina os modos de endereçamento (que inclui uma consideração dos registradores) e os formatos de instrução. O Capítulo 13 examina o computador com conjunto de instruções reduzido (RISC). A arquitetura RISC põe em dúvida muitas das decisões de projeto do conjunto de instruções tradicionalmente feitas nos computadores comerciais.



10.2 Tipos de operandos

As instruções de máquina operam sobre dados. As categorias gerais de dados mais importantes são:

- Endereços.
- Números.
- Caracteres.
- Dados lógicos.

Veremos, ao discutir os modos de endereçamento no Capítulo 11, que os endereços são, de fato, uma forma de dados. Em muitos casos, alguns cálculos precisam ser realizados sobre a referência do operando em uma instrução para determinar o endereço da memória principal ou virtual. Nesse contexto, os endereços podem ser considerados como inteiros sem sinal.

Outros tipos de dados comuns são números, caracteres e dados lógicos, e cada um destes é examinado rapidamente nesta seção. Além disso, algumas máquinas definem tipos de dados ou estruturas de dados especializadas. Por exemplo, pode haver operações de máquina que operam diretamente sobre uma lista ou uma *string* de caracteres.



Números

Todas as linguagens de máquina incluem tipos de dados numéricos. Até mesmo no processamento de dados não numéricos, existe a necessidade de os números atuarem como contadores, tamanhos de campo e assim por diante. Uma distinção importante entre números usados na matemática comum e números armazenados em um computador é que estes últimos são limitados. Isso é verdade em dois sentidos. Primeiro, existe um limite para a magnitude dos números representáveis em uma máquina e, segundo, no caso dos números de ponto flutuante, um limite em sua precisão. Assim, o programador precisa entender as consequências do arredondamento, do *overflow* e do *underflow*.

Três tipos de dados numéricos são comuns nos computadores:

- Inteiros binários ou ponto fixo binário.
- Ponto flutuante binário.
- Decimal.

Examinamos os dois primeiros com alguns detalhes no Capítulo 9. Resta-nos dizer algumas palavras sobre os números decimais.

Embora internamente todas as operações do computador sejam binárias em natureza, os usuários humanos do sistema lidam com números decimais. Assim, existe a necessidade de converter de decimal para binário na entrada e de binário para decimal na saída. Para aplicações onde existem muitas E/S e, comparadamente, poucos e simples cálculos, é preferível armazenar e operar os números em forma decimal. A representação mais comum para essa finalidade é **decimal agrupado** (*packed decimal*).¹

¹ Os livros-texto normalmente referem-se a isso como decimal codificado em binário (BCD, do inglês *binary coded decimal*). Estritamente falando, o código BCD refere-se à codificação de cada dígito decimal por uma sequência exclusiva de 4 bits. Decimal agrupado refere-se ao armazenamento de dígitos codificados em BCD usando um byte para cada dois dígitos.

Com o decimal agrupado, cada dígito decimal é representado por um código de 4 bits, no modo óbvio, com dois dígitos armazenados por byte. Assim, 0 = 0000, 1 = 0001, ..., 8 = 1000 e 9 = 1001. Observe que esse é um código ineficaz, pois somente 10 dos 16 valores possíveis em 4 bits são utilizados. Para formar números, códigos de 4 bits são enfileirados, normalmente em múltiplos de 8 bits. Assim, o código para 246 é 0000 0010 0100 0110. Esse código certamente é menos compacto que a representação binária direta, mas evita o *overhead* da conversão. Números negativos podem ser representados incluindo-se um dígito de sinal de 4 bits à esquerda ou à direita da sequência de dígitos decimais agrupados. Os valores de sinal padrão são 1100 para positivo (+) e 1101 para negativo (–).

Muitas máquinas oferecem instruções aritméticas para realizar operações diretamente sobre números decimais agrupados. Os algoritmos são muito semelhantes àqueles descritos na Seção 9.3, mas precisam considerar a operação de *carry* decimal.



Caracteres

Uma forma de dado comum é o texto, ou *strings* de caracteres. Embora os dados textuais sejam mais convenientes para os seres humanos, eles não podem, em forma de caracteres, serem facilmente armazenados ou transmitidos por sistemas de processamento de dados e comunicações. Esses sistemas são projetados para dados binários. Assim, diversos códigos foram elaborados, nos quais os caracteres são representados por uma sequência de bits. Talvez o exemplo comum mais antigo seja o código Morse. Hoje, o código de caracteres mais utilizado é o *International Reference Alphabet* (IRA), mais conhecido como *American Standard Code for Information Interchange* (ASCII; veja Apêndice F). Cada caractere nesse código é representado por um padrão exclusivo de 7 bits; assim, 128 caracteres diferentes podem ser representados. Esse é um número maior do que é necessário para representar os caracteres imprimíveis, e alguns dos padrões representam caracteres de *controle*. Alguns desses caracteres de controle têm a ver com o controle da impressão dos caracteres em uma página. Outros tratam de procedimentos de comunicação. Os caracteres codificados em IRA quase sempre são armazenados e transmitidos usando 8 bits por caractere. O oitavo bit pode ser definido como 0 ou usado como um bit de paridade, para detectar erros. Nesse último caso, o bit é definido de modo que o número total de 1s binários em cada octeto seja sempre ímpar (paridade ímpar) ou sempre par (paridade par).

Observe, na Tabela F.1 (Apêndice F) que, para o padrão de bits IRA 011XXXX, os dígitos de 0 a 9 são representados por seus equivalentes binários, 0000 a 1001, nos 4 bits mais à direita. Esse é o mesmo código do decimal agrupado. Isso facilita a conversão entre IRA de 7 bits e a representação decimal agrupada de 4 bits.

Outro código usado para codificar caracteres é o *Extended Binary Coded Decimal Interchange Code* (EBCDIC). EBCDIC é usado em mainframes IBM. Este é um código de 8 bits. Assim como IRA, EBCDIC é compatível com decimal agrupado. No caso do EBCDIC, os códigos de 11110000 a 11111001 representam os dígitos de 0 a 9.



Dados lógicos

Normalmente, cada palavra ou outra unidade endereçável (byte, meia-palavra e assim por diante) é tratada como uma única unidade de dados. Porém, às vezes é útil considerar uma unidade de n bits como consistindo em n itens de dados de 1 bit, com cada item tendo o valor 0 ou 1. Quando os dados são vistos dessa forma, eles são considerados como sendo dados *lógicos*.

Existem duas vantagens na visão orientada a bits. Primeiro, às vezes podemos querer armazenar um *array* de itens de dados booleanos ou binários, em que cada item pode assumir apenas os valores 1 (verdadeiro) e 0 (falso). Com dados lógicos, a memória pode ser usada de modo mais eficiente para esse armazenamento. Segundo, existem ocasiões em que queremos manipular os bits de um item de dados. Por exemplo, se as operações de ponto flutuante forem implementadas em software, precisamos ser capazes de deslocar bits significativos em algumas operações. Outro exemplo: para converter de IRA para decimal agrupado, precisamos extrair os 4 bits mais à direita de cada byte.

Observe que, nos exemplos anteriores, os mesmos dados são tratados às vezes como lógicos e outras vezes como numéricos ou texto. O “tipo” de uma unidade de dados é determinado pela operação que está sendo realizada sobre ele. Embora isso normalmente não aconteça em linguagens de alto nível, quase sempre acontece com a linguagem de máquina.

10.3 Tipos de dados Intel x86 e do ARM

Tipos de dados do x86

O x86 pode lidar com tipos de dados de 8 (byte), 16 (palavra), 32 (palavras duplas – *doubleword*), 64 (quatro palavras – *quadword*) e 128 (*double quadword*) bits de extensão. Para permitir o máximo de flexibilidade nas estruturas de dados e utilização de memória eficiente, as palavras não precisam ser alinhadas em endereços de número par; *palavras duplas* não precisam ser alinhadas em endereços divisíveis uniformemente por 4; e *quadwords* não precisam ser alinhadas em endereços divisíveis uniformemente por 8, e assim por diante. Porém, quando os dados são acessados por um barramento de 32 bits, as transferências de dados ocorrem em unidades de *palavras duplas*, começando em endereços divisíveis por 4. O processador converte a requisição para valores desalinhados em uma sequência de solicitações para a transferência do barramento. Assim como todas as máquinas Intel 80x86, o x86 usa o estilo *little-endian*; ou seja, o byte menos significativo é armazenado no endereço mais baixo (veja no Apêndice 10B uma discussão sobre os estilos de *endian*).

Byte, word, *doubleword*, *quadword* e *double quadword* são chamados de tipos de dados gerais. Além disso, o x86 admite um conjunto impressionante de tipos de dados específicos, que são reconhecidos e operados por instruções em particular. A Tabela 10.2 resume esses tipos.

Tabela 10.2 Tipos de dados x86

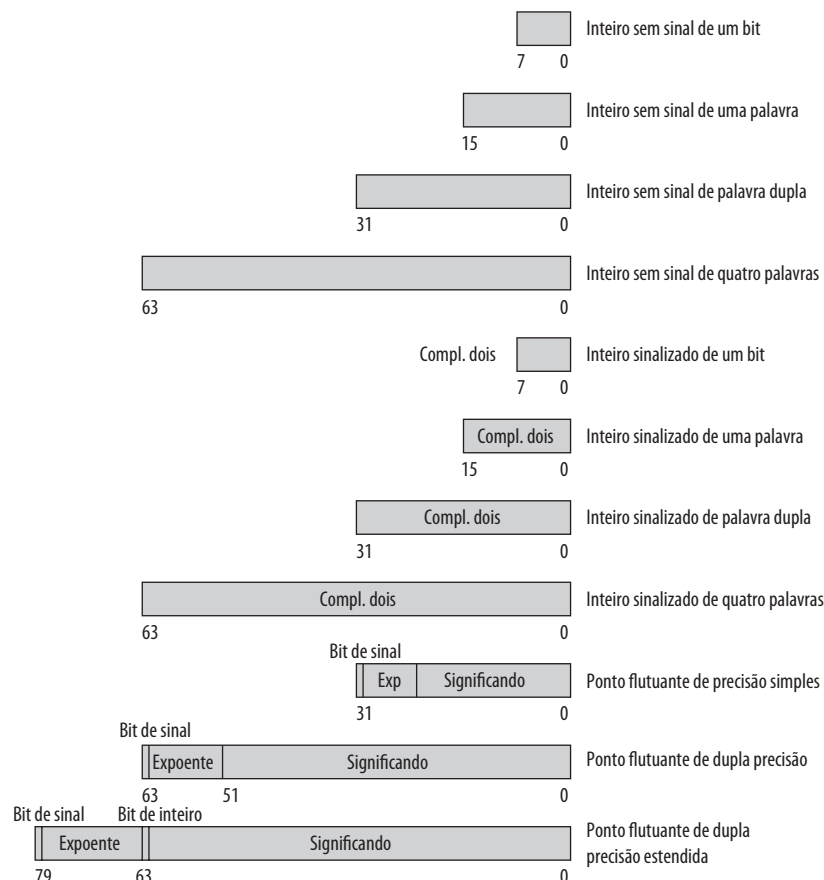
Tipo de dados	Descrição
Geral	Locais de byte, palavra (16 bits), palavras duplas (32 bits), quatro palavras (64 bits) e quatro palavras duplas (128 bits) com conteúdo binário arbitrário.
Números inteiros	Um valor binário com sinal, contido em um byte, palavra ou palavras duplas, usando a representação de complemento a dois.
Números ordinais	Um inteiro sem sinal contido em um byte, palavra ou palavras duplas.
Números em BCD (<i>Binary coded decimal</i>) (BCD) não agrupado	Uma representação de um dígito BCD no intervalo de 0 a 9, com um dígito em cada byte.
Agrupado BCD	Representação de byte agrupado de dois dígitos BCD; valor no intervalo de 0 a 99.
Ponteiro near	Um endereço efetivo de 16, 32 ou 64 bits, que representa o deslocamento dentro de um segmento. Usado para todos os ponteiros em uma memória não segmentada e para referências dentro de um segmento em uma memória segmentada.
Ponteiro far	Um endereço lógico consistindo em um seletor de segmento de 16 bits e um deslocamento de 16, 32 ou 64 bits. Ponteiros far são usados para referência à memória em um modelo de memória segmentado, onde a identidade de um segmento sendo acessado precisa ser especificada explicitamente.
Campo de bits	Uma sequência contígua de bits em que a posição de cada bit é considerada como uma unidade independente. Uma <i>string</i> de bits pode começar em qualquer posição de bit de qualquer byte e pode conter até 32 bits.
Cadeia de bits	Uma sequência contígua de bits, contendo de zero a $2^{32} - 1$ bits.
Cadeia de bytes	Uma sequência contígua de bytes, palavras ou <i>doublewords</i> , contendo de zero a $2^{32} - 1$ bytes.
Ponto flutuante	Ver Figura 10.4.
SIMD agrupada (do inglês <i>single instruction, multiple data</i> — única instrução, múltiplos dados)	Tipos de dados de 64 e 128 bits agrupados.

A Figura 10.4 ilustra os tipos de dados numéricos do x86. Os números inteiros com sinal estão em representação de complemento a dois e podem ter 16, 32 ou 64 bits de extensão. Os em ponto flutuante na realidade referem-se a um conjunto de tipos que são usados pela unidade de ponto flutuante e operados por instruções de ponto flutuante. As três representações de ponto flutuante se ajustam ao padrão IEEE 754.

Os tipos de dados SIMD (única instrução, múltiplos dados) agrupados foram introduzidos à arquitetura x86 como parte das extensões do conjunto de instruções para otimizar o desempenho de aplicações de multimídia. Essas extensões incluem MMX (Multimedia Extensions) e SSE (Streaming SIMD Extensions). O conceito básico é que múltiplos operandos são agrupados em um único item de memória referenciado e que esses múltiplos operandos são operados em paralelo. Os tipos de dados são os seguintes:

- **Byte agrupado e inteiro de byte agrupado:** bytes agrupados em uma *quadword* de 64 bits ou *double quadword* de 128 bits, interpretada como um campo de bit ou como um inteiro.
- **Palavra agrupada e inteiro de palavra agrupada:** palavras de 16 bits agrupados em uma *quadword* de 64 bits ou *double quadword* de 128 bits, interpretada como um campo de bit ou como um inteiro.
- **Doubleword agrupado e inteiro de doubleword agrupado:** *doublewords* de 32 bits agrupados em uma *quadword* de 64 bits ou *double quadword* de 128 bits, interpretada como um campo de bit ou como um inteiro.
- **Quadword agrupado e inteiro de quadword agrupado:** duas *quadwords* de 64 bits agrupadas em uma *double quadword* de 128 bits, interpretada como um campo de bit ou como um inteiro.
- **Ponto flutuante de precisão simples agrupado e ponto flutuante de precisão dupla agrupado:** quatro valores de ponto flutuante de 32 bits ou dois valores de ponto flutuante de 64 bits agrupados em uma *double quadword* de 128 bits.

Figura 10.4 Formatos de dados numéricos x86





Tipos de dados do ARM

Processadores ARM admitem tipos de dados de 8 (byte), 16 (meia-palavra) e 32 (palavra) bits de extensão. Normalmente, o acesso de halfword precisa ser alinhado por halfword e os acessos de palavra precisam ser alinhados por palavra. Para tentativas de acesso desalinhadas, a arquitetura admite três alternativas.

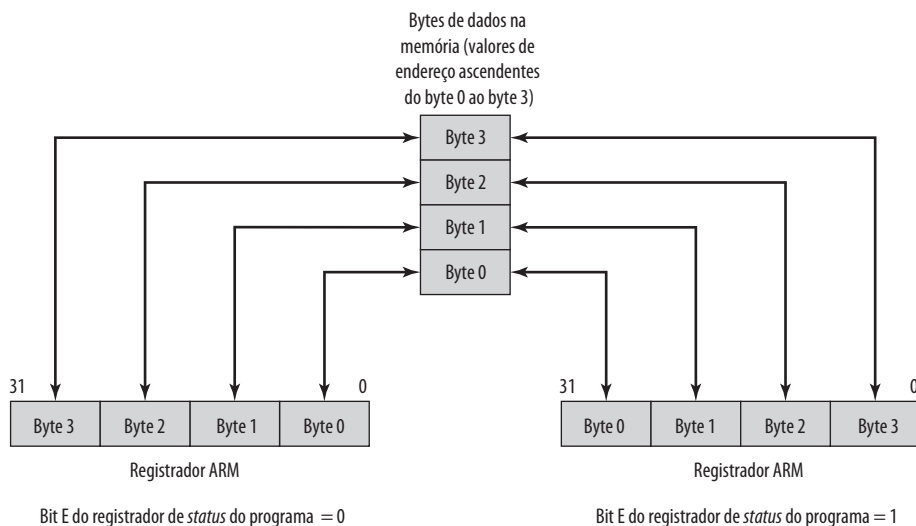
- Caso padrão:
 - O endereço é tratado como truncado, com os bits de endereço [1:0] tratados como zero para acesso por palavra, e o bit de endereço [0] tratado como zero para acesso por meia-palavra.
 - Instruções ARM de única palavra são arquiteturalmente definidas para girar à direita os dados alinhados por palavra transferidos por um endereço não alinhado por palavra de um, dois ou três palavras, dependendo do valor dos dois bits de endereço menos significativos.
- **Verificação de alinhamento:** quando o bit de controle apropriado é definido, um sinal de abortar dados indica uma falha de alinhamento para a tentativa de acesso desalinhado.
- **Acesso desalinhado:** quando essa opção é habilitada, o processador usa um ou mais acessos à memória para gerar a transferência exigida de bytes adjacentes de modo transparente ao programador.

Para os três tipos de dados (byte, meia-palavra e palavra), uma interpretação sem sinal é admitida, em que o valor representa um inteiro sem sinal, não negativo. Os três tipos de dados também podem ser usados para os inteiros com sinal em complemento de dois.

A maioria das implementações de processador ARM não oferece hardware de ponto flutuante, o que economiza energia e área. Se a aritmética de ponto flutuante for exigida em tais processadores, ela deverá ser implementada em software. O ARM admite um coprocessador de ponto flutuante opcional, que aceita os tipos de dados de ponto flutuante de precisão simples e dupla, definidos no IEEE 754.

SUPORTE PARA ENDIAN Um bit de estado (bit E) no registrador de controle do sistema é marcado e apagado sob controle do programa, usando a instrução SETEND. O bit E define qual modo será usado *endian* para ler e armazenar dados. A Figura 10.5 ilustra a funcionalidade associada ao bit E para uma operação load ou store de uma palavra. Esse mecanismo permite o *load/store* eficiente de dados dinâmicos para projetistas de sistemas que sabem que precisam acessar estruturas de dados no tipo de *endian* oposto ao seu sistema operacional/ambiente. Observe que o endereço de cada byte de dados é fixo na memória. Porém, a ordem de bytes em um registrador é diferente.

Figura 10.5 Suporte a *endian* no ARM — Load/Store de palavra com o bit E





10.4 Tipos de operações

O número de *opcodes* diferentes varia bastante de uma máquina para outra. Porém, os mesmos tipos gerais de operações são encontrados em todas as máquinas. Uma categorização útil e típica é a seguinte:

- Transferência de dados.
- Aritmética.
- Lógica.
- Conversão.
- E/S.
- Controle do sistema.
- Transferência de controle.

A Tabela 10.3 (baseada em Hayes, 1998^a) lista tipos de instrução comuns em cada categoria. Esta seção oferece um breve estudo desses diversos tipos de operações, junto com uma breve discussão das ações tomadas pelo processador para executar determinado tipo de operação (resumidas na Tabela 10.4). Esse último tópico é examinado com mais detalhes no Capítulo 12.



Transferência de dados

O tipo mais fundamental de instrução de máquina é a instrução de transferência. A instrução de transferência de dados precisa especificar várias coisas. Em primeiro lugar, o local dos operandos de origem e de destino. Cada

Tabela 10.3 Operações comuns do conjunto de instruções

Tipo	Nome da operação	Descrição
Transferência de dados	Move (transferência)	Transfere palavra ou bloco da origem ao destino
	Store (armazenar)	Transfere palavra do processador para a memória
	Load (carregar)	Transfere palavra da memória para o processador
	Exchange	Troca o conteúdo da origem e do destino
	Clear (reset)	Transfere palavra de 0s para o destino
	Set	Transfere palavra de 1s para o destino
	Push	Transfere palavra da origem para o topo da pilha
	Pop	Transfere palavra do topo da pilha para o destino
Aritmética	Add	Calcula a soma de dois operandos
	Subtract	Calcula a diferença de dois operandos
	Multiply	Calcula o produto de dois operandos
	Divide	Calcula o quociente de dois operandos
	Absolute	Substitui o operando pelo seu valor absoluto
	Negate	Troca o sinal do operando
	Increment	Soma 1 ao operando
	Decrement	Subtrai 1 do operando

(Continua)

Tabela 10.3 Operações comuns do conjunto de instruções (continuação)

Tipo	Nome da operação	Descrição
Lógica	AND	Realiza o AND lógico
	OR	Realiza o OR lógico
	NOT (complemento)	Realiza o NOT lógico
	Exclusive-OR	Realiza o XOR lógico
	Test	Testa condição especificada; define flag(s) com base no resultado
	Compare	Faz comparação lógica ou aritmética de dois ou mais operandos; define flag(s) com base no resultado
	Set control variables	Classe de instruções para definir controles para fins de proteção, tratamento de interrupção, controle de tempo etc.
	Shift	Desloca o operando para a esquerda (direita), introduzindo constantes na extremidade
	Rotate	Desloca ciclicamente o operando para a esquerda (direita), de uma extremidade à outra
Transferência de controle	Jump (desvio)	Transferência incondicional; carrega PC com endereço especificado
	Jump conditional	Testa condição especificada; ou carrega PC com endereço especificado ou não faz nada, com base na condição
	Jump to subroutine	Coloca informação do controle do programa atual em local conhecido; salta para endereço especificado
	Return	Substitui conteúdo do PC por outro registrador de local conhecido
	Execute	Busca operando do local especificado e executa como instrução; não modifica o PC
	Skip	Incrementa o PC para saltar para a próxima instrução
	Skip conditional	Testa condição especificada; ou salta ou não faz nada, com base na condição
	Halt	Termina a execução do programa
	Wait (hold)	Termina a execução do programa; testa condição especificada repetidamente; retoma a execução quando a condição for satisfeita
	No operation	Nenhuma operação é realizada, mas a execução do programa continua
Entrada/saída	Input (leitura)	Transfere dados da porta de E/S ou dispositivo especificado para o destino (por exemplo, memória principal ou registrador do processador)
	Output (escrita)	Transfere dados da origem especificada para porta de E/S ou dispositivo
	Start I/O	Transfere instruções para o processador de E/S para iniciar operação de E/S
	Test I/O	Transfere informações de <i>status</i> do sistema de E/S para destino especificado
Conversão	Translate	Traduz valores em uma seção da memória com base em uma tabela de correspondências
	Convert	Converte o conteúdo de uma palavra de uma forma para outra (por exemplo, decimal agrupado para binário)

local pode ser memória, um registrador ou o topo da pilha. Segundo, a extensão dos dados a serem transferidos precisa ser indicada. Terceiro, assim como todas as instruções com operandos, o modo de endereçamento para cada operando precisa ser especificado. Esse último ponto é discutido no Capítulo 11.

Tabela 10.4 Ações do processador para diversos tipos de operação

Transferência de dados	Transfere dados de um local para outro
	Se a memória estiver envolvida:
	Determina o endereço da memória
	Realiza transformação de endereço de memória virtual para real
	Verifica cache
Aritmética	Inicia leitura/escrita da memória
	Pode envolver transferência de dados, antes e/ou depois
	Realiza função na ALU
Lógica	Define códigos de condição e flags
Conversão	O mesmo que aritmética
Transferência de controle	Semelhante a aritmética e lógica. Pode envolver lógica especial para realizar conversão
E/S	Atualiza contador de programa. Para chamada/retorno de sub-rotina, gerencia passagem e ligação de parâmetros
	Emita comando para módulo de E/S
	Se E/S mapeada na memória, determina o endereço mapeado na memória

A escolha das instruções de transferência de dados a incluir em um conjunto de instruções exemplifica os tipos de escolhas que o projetista precisa tomar. Por exemplo, o local geral (memória ou registrador) de um operando pode ser indicado na especificação do *opcode* ou no operando. A Tabela 10.5 mostra exemplos das instruções de transferência de dados do IBM EAS/390 mais comuns. Observe que existem variantes para indicar a quantidade de dados a serem transferidos (8, 16, 32 ou 64 bits). Além disso, existem diferentes instruções para transferências registrador para registrador, registrador para memória, memória para registrador e memória para memória. Ao contrário, o VAX tem uma instrução de movimentação (MOV) com variantes para diferentes quantidades de dados a serem movidos, mas especifica se um operando é registrador ou memória como parte do operando. A técnica do VAX é um pouco mais fácil para o programador, que precisa lidar com menos mnemônicos. Porém, ela também é menos compacta que a técnica do IBM EAS/390, pois

Tabela 10.5 Exemplos de operações de transferência de dados IBM EAS/390

Mnemônico da operação	Nome	Número de bits transferidos	Descrição
L	Load	32	Transfere de memória a registrador
LH	Load Halfword	16	Transfere de memória a registrador
LR	Load	32	Transfere de registrador a registrador
LER	Load (Short)	32	Transfere de registrador de ponto flutuante a registrador de ponto flutuante
LE	Load (Short)	32	Transfere de memória a registrador de ponto flutuante
LDR	Load (Long)	64	Transfere de registrador de ponto flutuante a registrador de ponto flutuante
LD	Load (Long)	64	Transfere de memória a registrador de ponto flutuante
ST	Store	32	Transfere de registrador a memória
STH	Store Halfword	16	Transfere de registrador a memória
STC	Store Character	8	Transfere de registrador a memória
STE	Store (Short)	32	Transfere de registrador de ponto flutuante a memória
STD	Store (Long)	64	Transfere de registrador de ponto flutuante a memória

o local (registrador *versus* memória) de cada operando deve ser especificado separadamente na instrução. Vamos voltar a essa diferença quando discutirmos sobre os formatos de instrução, no próximo capítulo.

Em termos de ação do processador, as operações de transferência de dados talvez sejam o tipo mais simples. Se a origem e o destino forem registradores, então o processador simplesmente faz com que os dados sejam transferidos de um registrador para outro; essa é uma operação interna ao processador. Se um ou ambos operandos estiverem na memória, então o processador deve realizar algumas ou todas as seguintes ações:

1. Calcular o endereço de memória, com base no modo de endereço (discutido no Capítulo 11).
2. Se o endereço se referir à memória virtual, traduzir de endereço virtual para real.
3. Determinar se o item endereçado está na cache.
4. Se não, emitir um comando para o módulo de memória.



Aritméticas

A maioria das máquinas oferece as operações aritméticas básicas de adição, subtração, multiplicação e divisão. Estas são invariavelmente fornecidas para números inteiros com sinal (ponto fixo). Normalmente, elas também são fornecidas para números de ponto flutuante e decimal agrupado.

Outras operações possíveis incluem uma série de instruções de único operando, por exemplo:

- **Absolute:** apanha o valor absoluto do operando.
- **Negate:** inverte o sinal do operando.
- **Increment:** soma 1 ao operando.
- **Decrement:** subtrai 1 do operando.

A execução de uma instrução aritmética pode envolver operações de transferência de dados para posicionar operandos para entrada na ALU, e entregar a saída da ALU. A Figura 3.5 ilustra as movimentações envolvidas nas operações de transferência de dados e aritméticas. Além disso, naturalmente, a parte da ALU do processador realiza a operação desejada.



Lógicas

A maioria das máquinas também oferece uma série de operações para manipular bits individuais de uma palavra ou outras unidades endereçáveis, normalmente conhecidas como "*bit twiddling*". Elas são baseadas em operações booleanas (veja Capítulo 20).

Algumas das operações lógicas básicas que podem ser realizadas sobre dados booleanos ou binários aparecem na Tabela 10.6. A operação NOT inverte um bit. AND, OR e Exclusive-OR (XOR) são as funções lógicas mais comuns com dois operandos. EQUAL é um teste binário útil.

Essas operações lógicas podem ser aplicadas bit a bit a unidades de dados lógicas de n bits. Assim, se dois registradores contêm os dados

(R1) = 10100101

(R2) = 00001111

Tabela 10.6 Operações lógicas básicas

P	Q	NOT P	P AND Q	Q OR Q	P XOR Q	P = Q
0	0	1	0	0	0	1
0	1	1	0	1	1	0
1	0	0	0	1	1	0
1	1	0	1	1	0	1

então,

$$(R1) \text{ AND } (R2) = 00000101$$

onde a notação (X) significa o conteúdo do local X. Assim, a operação AND pode ser usada como uma *máscara* que seleciona certos bits em uma palavra e zeros dos bits restantes. Como outro exemplo, se dois registradores contêm

$$(R1) = 10100101$$

$$(R2) = 11111111$$

então,

$$(R1) \text{ XOR } (R2) = 01011010$$

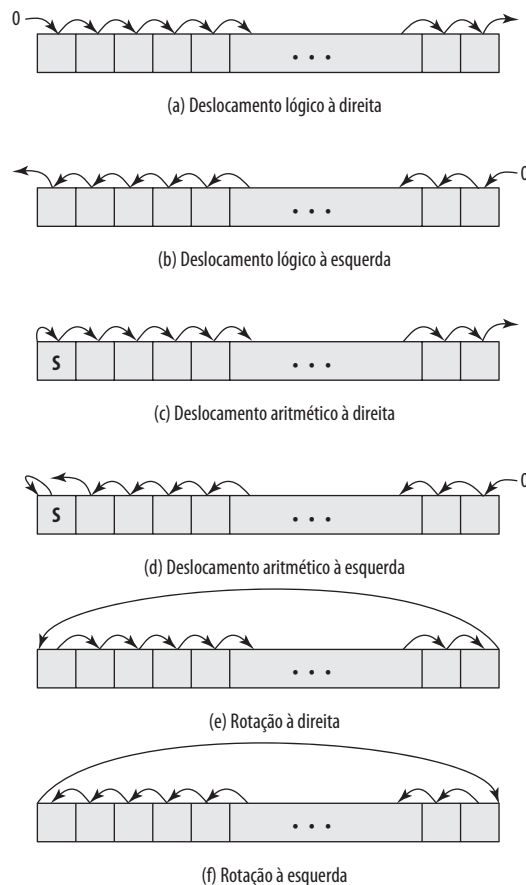
Com uma palavra definida como todos 1s, a operação XOR inverte todos os bits na outra palavra (complemento a um).

Além das operações lógicas bit a bit, a maioria das máquinas oferece uma série de funções de deslocamento e rotação. As operações mais básicas são ilustradas na Figura 10.6. Com um **deslocamento lógico**, os bits de uma palavra são deslocados para a esquerda ou para a direita. Em uma extremidade, o bit deslocado para fora se perde. Na outra extremidade, um 0 é deslocado para dentro. Os deslocamentos lógicos são úteis principalmente para isolar campos dentro de uma palavra. Os 0s que são deslocados para dentro de uma palavra afastam informações indesejadas, que são deslocadas para fora pela outra extremidade.

Como um exemplo, suponha que queiramos transmitir caracteres de dados para um dispositivo de E/S 1 caractere de cada vez. Se cada palavra da memória tiver 16 bits de extensão e tiver dois caracteres, temos que *desagrupar* os caracteres antes que eles possam ser enviados. Para enviar os dois caracteres em uma palavra:

1. Carregue a palavra em um registrador.
2. Desloque para a direita oito vezes. Isso desloca o caractere restante para a metade direita do registrador.

Figura 10.6 Operações de deslocamento e rotação



3. Realize a E/S. O módulo de E/S lê os 8 bits de ordem mais baixa do barramento de dados.

As etapas anteriores resultam em enviar os caracteres da esquerda. Para enviar o caractere da direita:

1. Carregue a palavra novamente no registrador.
2. Faça o AND com 0000000011111111. Isso mascara o caractere à esquerda.
3. Realize a E/S.

A operação de **deslocamento aritmético** trata os dados como um inteiro com sinal e não desloca o bit de sinal. Em um deslocamento aritmético à direita, o bit de sinal é replicado para a posição de bit à sua direita. Em um deslocamento aritmético à esquerda, um deslocamento lógico à esquerda é realizado sobre todos os bits, menos o bit de sinal, que é retido. Essas operações podem agilizar certas operações aritméticas. Com números na notação de complemento de dois, um deslocamento aritmético à direita corresponde a uma divisão por 2, truncando números ímpares. Um deslocamento aritmético à esquerda e um deslocamento lógico à esquerda correspondem a uma multiplicação por 2 quando não existe *overflow*. Se houver *overflow*, as operações de deslocamento aritmético e lógico à esquerda produzem diferentes resultados, mas o deslocamento aritmético à esquerda retém o sinal do número. Devido ao potencial para *overflow*, muitos processadores não incluem essa instrução, incluindo o PowerPC e o Itanium. Outros, como o IBM EAS/390, oferecem a instrução. Curiosamente, a arquitetura x86 inclui um deslocamento aritmético à esquerda, mas o define como sendo idêntico a um deslocamento lógico à esquerda.

As operações de **rotação**, ou deslocamento cíclico, preservam todos os bits como operando. Um uso de uma rotação é para trazer cada bit sucessivamente para o bit mais à esquerda, onde pode ser identificado testando o sinal do dado (tratado como um número).

Assim como as operações aritméticas, as operações lógicas envolvem a atividade da ALU e podem envolver operações de transferência de dados. A Tabela 10.7 oferece exemplos de todas as operações de deslocamento e rotação discutidas nesta subseção.



Conversão

Instruções de conversão são aquelas que mudam o formato ou operam sobre o formato dos dados. Um exemplo é a conversão de decimal para binário. Um exemplo de uma instrução de edição mais complexa é a instrução Translate (TR) do EAS/390. Essa instrução pode ser usada para converter de um código de 8 bits para outro, e utiliza três operandos:

TR R1 (L), R2

O operando R2 contém o endereço do início de uma tabela de códigos de 8 bits. Os L bytes começando no endereço especificado em R1 são traduzidos, cada byte sendo substituído pelo conteúdo de uma entrada de tabela indexada por esse byte. Por exemplo, para traduzir de EBCDIC para IRA, primeiro criamos uma tabela de 256 bytes nos locais de armazenamento, digamos, 1000-10FF hexadecimal. A tabela contém os caracteres do código IRA na sequência da representação binária do código EBCDIC; ou seja, o código IRA é colocado na tabela no local relativo igual ao valor binário do código EBCDIC do mesmo caractere. Assim, os locais 10F0 a 10F9 terão os valores de 30 a 39, pois F0 é o código EBCDIC para o dígito 0, e 30 é o código IRA para o dígito 0, e assim por diante até o dígito 9.

Tabela 10.7 Exemplos de operações de deslocamento e rotação

Input	Operação	Resultado
10100110	Deslocamento lógico à direita (3 bits)	00010100
10100110	Deslocamento lógico à esquerda (3 bits)	00110000
10100110	Deslocamento aritmético à direita (3 bits)	11110100
10100110	Deslocamento aritmético à esquerda (3 bits)	10110000
10100110	Rotação à direita (3 bits)	11010100
10100110	Rotação à esquerda (3 bits)	00110101

Agora, suponha que tenhamos o EBCDIC para os dígitos 1984 começando no local 2100 e queiramos traduzir para IRA. Considere o seguinte:

- Locais 2100–2103 contêm F1 F9 F8 F4.
- R1 contém 2100.
- R2 contém 1000.

Então, se executarmos

TR R1 (4), R2

os locais 2100–2103 terão 31 39 38 34.



Entrada/saída

As instruções de entrada/saída foram discutidas com alguns detalhes no Capítulo 7. Como vimos, existem diversas técnicas que podem ser usadas, incluindo E/S programada independente, E/S programada mapeada na memória, DMA e o uso de um processador de E/S. Muitas implementações oferecem apenas algumas instruções de E/S, com ações específicas ditadas por parâmetros, códigos ou palavras de comando.



Controle do sistema

As instruções de controle do sistema são aquelas que podem ser executadas apenas enquanto o processador está em um certo estado privilegiado ou está executando um programa em uma área privilegiada especial da memória. Normalmente, essas instruções são reservadas para o uso do sistema operacional.

Alguns exemplos de operações de controle do sistema são os seguintes. Uma instrução de controle do sistema pode ler ou alterar um registrador de controle; discutimos os registradores de controle no Capítulo 12. Outro exemplo é uma instrução para ler ou modificar uma chave de proteção de armazenamento, como a que é usada no sistema de memória do EAS/390. Outro exemplo é o acesso para processar blocos de controle em um sistema multiprogramado.



Transferência de controle

Para todos os tipos de operação discutidos até aqui, a próxima instrução a ser realizada é aquela que, na memória, vem imediatamente após a instrução atual. Porém, uma fração significativa das instruções em um programa tem como função mudar a sequência de execução de instruções. Para essas instruções, a operação realizada pelo processador é atualizar o contador de programa para conter o endereço de alguma instrução na memória.

Existem vários motivos pelos quais as operações de transferência de controle são necessárias. Entre os mais importantes estão os seguintes:

1. No uso prático dos computadores, é essencial poder executar cada instrução mais de uma vez e, talvez, muitas milhares de vezes. Podem ser necessárias milhares ou talvez milhões de instruções para implementar uma aplicação. Seria impensável se cada instrução tivesse que ser escrita separadamente. Se uma tabela ou uma lista de itens tiver que ser processada, um loop de programa é necessário. Uma sequência de instruções é executada repetidamente para processar todos os dados.
2. Praticamente todos os programas envolvem alguma tomada de decisão. Gostaríamos que o computador fizesse uma coisa se uma condição for verdadeira, e outra coisa se outra condição for verdadeira. Por exemplo, uma sequência de instruções calcula a raiz quadrada de um número. No início da sequência, o sinal do número é testado. Se o número for negativo, o cálculo não é realizado, mas uma condição de erro é informada.
3. Compor corretamente um programa de computador de tamanho grande, ou mesmo médio, é uma tarefa extremamente difícil. É útil que haja mecanismos para dividir a tarefa em pedaços menores, que possam ser trabalhados um de cada vez.

Agora, vamos passar a uma discussão das operações de transferência de controle mais comuns encontradas nos conjuntos de instruções: desvio, salto e chamada de procedimento.

INSTRUÇÕES DE DESVIO Uma instrução de desvio, também chamada de instrução de salto, tem como um de seus operandos o endereço da próxima instrução a ser executada. Normalmente, é uma instrução de **desvio condicional**, ou seja, o desvio é tomado (atualizar o contador de programa para que seja igual ao endereço especificado no operando) somente se a condição for atendida. Caso contrário, a próxima instrução na sequência é executada

(incrementar contador de programa normalmente). Uma instrução de desvio em que o desvio sempre é tomado é um **desvio incondicional**.

Existem duas maneiras comuns de gerar a condição a ser testada em uma instrução de desvio condicional. Primeiro, a maioria das máquinas oferece um código de condição de 1 ou mais bits, que é definido como o resultado de algumas operações. Esse código pode ser imaginado como um registrador pequeno, visível ao usuário. Como um exemplo, uma operação aritmética (ADD, SUBTRACT e assim por diante) poderia definir um código de condição de 2 bits com um dos quatro valores a seguir: 0, positivo, negativo, *overflow*. Em tal máquina, poderia haver quatro instruções de desvio condicional diferentes:

BRP X Desvia para local X se resultado for positivo.
BRN X Desvia para local X se resultado for negativo.
BRZ X Desvia para local X se resultado for zero.
BRO X Desvia para local X se houver *overflow*.

Em todos esses casos, o resultado referenciado é o resultado da operação mais recente que define o código de condição.

Outra técnica que pode ser usada com um formato de instrução de três endereços é realizar uma comparação e especificar um desvio na mesma interface. Por exemplo,

BRE R1, R2, X Desvia para X se conteúdo de R1 = conteúdo de R2.

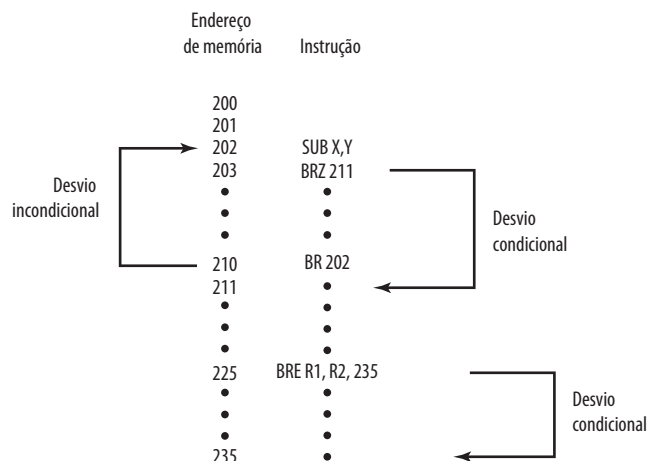
A Figura 10.7 mostra exemplos dessas operações. Observe que um desvio pode ser *para frente* (uma instrução com um endereço mais alto) ou *para trás* (endereço mais baixo). O exemplo mostra como um desvio incondicional e um desvio condicional podem ser usados para criar um loop repetitivo de instruções. As instruções nos locais de 202 a 210 serão executadas repetidamente, até que o resultado de subtrair Y de X seja 0.

INSTRUÇÕES DE SALTO (SKIP) Outra forma de instrução de transferência de controle é a instrução de salto. A instrução de salto inclui um endereço implícito. Normalmente, o salto implica que uma instrução seja pulada; assim, o endereço implícito é igual ao endereço da próxima instrução mais o tamanho de uma instrução.

Como a instrução de salto não exige um campo de endereço de destino, ela está livre para realizar outras coisas. Um exemplo típico é a instrução de incrementar e pular se for zero (ISZ — *Increment-and-Skip-if-Zero*). Considere o seguinte pedaço de programa:

```
301
.
.
.
309 ISZ R1
310 BR 301
311
```

Figura 10.7 Instruções de desvio



Nesse pedaço, as duas instruções de transferência de controle são usadas para implementar um loop iterativo. R1 é definido como o negativo do número de iterações a serem realizadas. Ao final do loop, R1 é incrementado. Se não for 0, o programa desvia de volta ao início do loop. Caso contrário, o desvio é pulado, e o programa continua com a próxima instrução após o final do loop.

INSTRUÇÕES DE CHAMADA DE PROCEDIMENTO Talvez a inovação mais importante no desenvolvimento de linguagens de programação seja o *procedimento*. Um procedimento é um programa de computação autocontido, que é incorporado em um programa maior. Em qualquer ponto no programa, o procedimento pode ser invocado, ou *chamado*. O processador é instruído a ir e executar o procedimento inteiro e depois retornar ao ponto onde ocorreu a chamada.

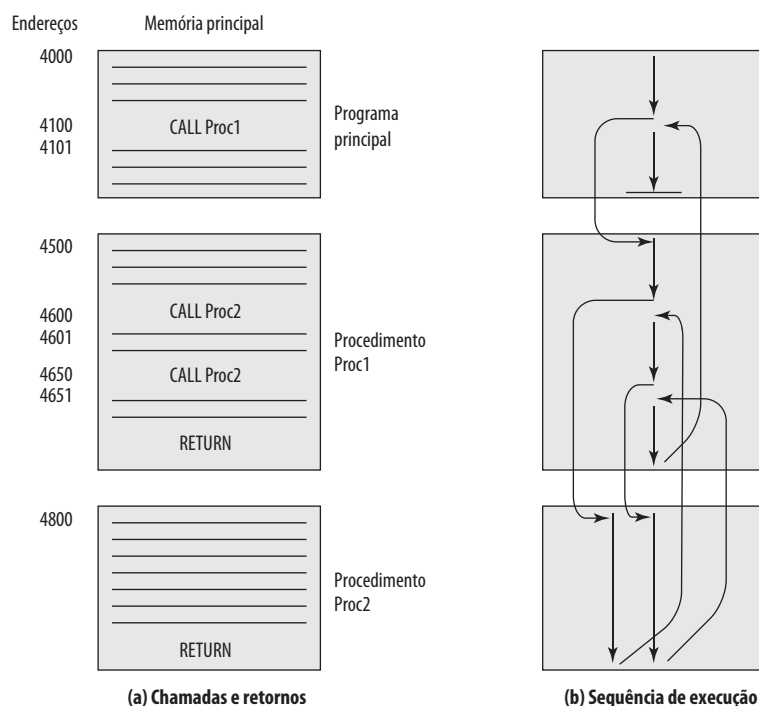
Os dois motivos principais para o uso de procedimentos são economia e modularidade. Um procedimento permite que o mesmo trecho de código seja usado muitas vezes. Isso é importante por economia no esforço de programação e para fazer um uso mais eficiente do espaço de armazenamento no sistema (o programa precisa ser armazenado). Os procedimentos também permitem que grandes tarefas de programação sejam subdivididas em unidades menores. Esse uso da *modularidade* facilita bastante a tarefa de programação.

O mecanismo de procedimento envolve duas instruções básicas: uma instrução de chamada que desvia do local atual para o procedimento, e uma instrução de retorno que retorna do procedimento para o local do qual ele foi chamado. Ambas são formas de instruções de desvio.

A Figura 10.8a ilustra o uso de procedimentos para construir um programa. Nesse exemplo, existe um programa principal começando no local 4000. Esse programa inclui uma chamada ao procedimento PROC1, começando no local 4500. Quando essa instrução de chamada é encontrada, o processador suspende a execução do programa principal e inicia a execução de PROC1 buscando a próxima instrução do local 4500. Dentro de PROC1, existem duas chamadas a PROC2 no local 4800. Em cada caso, a execução de PROC1 é suspensa e PROC2 é executado. A instrução RETURN faz com que o processador retorne ao programa que chamou e continue a execução na instrução após a instrução CALL correspondente. Esse comportamento é ilustrado na Figura 10.8b.

Três pontos precisam ser observados:

Figura 10.8 Procedimentos aninhados



1. Um procedimento pode ser chamado de mais de um local.
2. Uma chamada de procedimento pode aparecer em outro procedimento. Isso permite o *aninhamento* de procedimentos até uma profundidade qualquer.
3. Cada chamada de procedimento corresponde a um retorno no programa chamado.

Como gostaríamos de poder chamar um procedimento a partir de diversos pontos, o processador precisa, de alguma forma, salvar o endereço de retorno para que este possa ocorrer corretamente. Existem três locais comuns para armazenar o endereço de retorno:

- Registrador.
- Início do procedimento chamado.
- Topo da pilha.

Considere uma instrução em linguagem de máquina CALL X, que significa *chamar procedimento no local X*. Se a técnica de registrador for usada, CALL X causa as seguintes ações:

$$RN \leftarrow PC + \Delta$$

$$PC \leftarrow X$$

onde RN é um registrador que sempre é usado para essa finalidade, PC é o contador de programa e Δ é o tamanho da instrução. O procedimento chamado agora pode salvar o conteúdo de RN a ser usado para o retorno posterior.

Uma segunda possibilidade é armazenar o endereço de retorno no início do procedimento. Nesse caso, CALL X causa

$$X \leftarrow PC + \Delta$$

$$PC \leftarrow X + 1$$

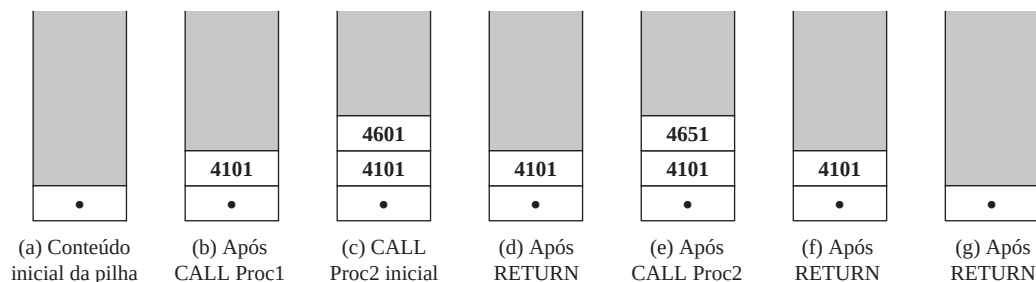
Isso é muito prático. O endereço de retorno foi armazenado de forma segura.

As duas técnicas anteriores funcionam e têm sido usadas. A única limitação dessas técnicas é que elas complicam o uso dos procedimentos *reentrantes*. Um procedimento reentrante é aquele em que é possível ter várias chamadas abertas ao mesmo tempo. Um procedimento recursivo (aquele que chama a si mesmo) é um exemplo do uso desse recurso (veja Apêndice H). Se os parâmetros forem passados por meio de registradores ou da memória para o procedimento reentrante, algum código deverá ser responsável por salvar os parâmetros, de modo que os registradores ou o espaço da memória estejam disponíveis para outras chamadas de procedimento.

Uma técnica mais geral e poderosa é usar uma pilha (veja, no Apêndice 10A, uma discussão sobre as pilhas). Quando o processador executa uma chamada, ele coloca o endereço de retorno na pilha. Quando ele executa um retorno, usa o endereço armazenado na pilha. A Figura 10.9 ilustra o uso da pilha.

Além de oferecer um endereço armazenado de retorno, normalmente também é preciso passar parâmetros com uma chamada de procedimento. Estes podem ser passados em registradores. Outra possibilidade é armazenar os parâmetros na memória logo após a instrução CALL. Nesse caso, o retorno precisa ser para o local após os parâmetros. Novamente, essas duas técnicas possuem desvantagens. Se forem usados registradores, o programa

Figura 10.9 Uso da pilha para implementar sub-rotinas aninhadas da Figura 10.8



chamado e o programa que chama precisam ser escritos para garantir que os registradores sejam usados devidamente. O armazenamento de parâmetros na memória dificulta a troca de um número variável de parâmetros. As duas técnicas impedem o uso de procedimentos reentrantes.

Uma técnica mais flexível para a passagem de parâmetros é a pilha. Quando o processador executa uma chamada, ele não apenas empilha o endereço de retorno, mas os parâmetros a serem passados ao procedimento chamado. O procedimento chamado pode acessar os parâmetros a partir da pilha. Ao retornar, os parâmetros de retorno também podem ser colocados na pilha. O conjunto inteiro de parâmetros, incluindo endereço de retorno, que é armazenado para uma chamada de procedimento é chamada de stack frame.

Um exemplo aparece na Figura 10.10. O exemplo refere-se ao procedimento P em que as variáveis locais x1 e x2 são declaradas, e o procedimento Q, que P pode chamar e no qual as variáveis locais y1 e y2 são declaradas. Nessa figura, o ponto de retorno para cada procedimento é o primeiro item armazenado no stack frame de pilha correspondente. Em seguida, é armazenado um ponteiro para o início do stack frame anterior. Isso é necessário se o número ou o tamanho dos parâmetros a serem empilhados for variável.

10.5 Tipos de operação Intel x86 e do ARM

Tipos de operação do x86

O x86 oferece uma complexa gama de tipos de operação, incluindo uma série de instruções especializadas. A intenção foi oferecer ferramentas para quem implementa compiladores, capazes de produzir códigos otimizados em linguagem de máquina a partir dos programas em linguagens de alto nível. A Tabela 10.8 lista os tipos e oferece exemplos de cada um. A maioria destes refere-se a instruções convencionais, encontradas na maioria dos conjuntos de instrução de máquina, mas vários tipos de instruções são ajustados à arquitetura x86 e são de interesse particular. O Apêndice A de Carter (1996^b) lista as instruções do x86, junto com os operandos para cada uma e o efeito da instrução sobre os códigos de condição. O Apêndice B do manual da linguagem de montagem NASM oferece uma descrição mais detalhada de cada instrução do x86. Os dois documentos estão disponíveis no site Web deste livro.

Figura 10.10 Crescimento do frame de pilha usando procedimentos de exemplo P e Q

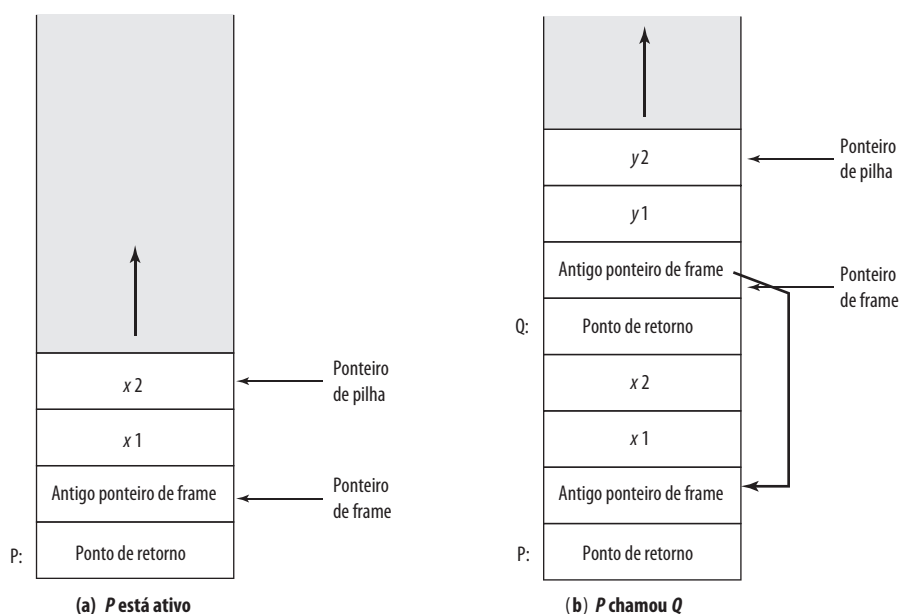


Tabela 10.8 Tipos de operação do x86 (com exemplos de operações típicas)

Instrução	Descrição
Movimentação de dados	
MOV	Operação de movimentação entre registradores ou entre registrador e memória.
PUSH	Coloca operando na pilha.
PUSHA	Coloca todos os registradores na pilha.
MOVSX	Move byte, palavra, palavra dupla, com extensão de sinal. Move um byte para uma palavra, ou uma palavra para uma palavra dupla, com extensão de sinal para complemento de dois.
LEA	Carrega endereço efetivo. Carrega o deslocamento do operando de origem, em vez do seu valor, ao operando de destino.
XLAT	Tradução de tabela de pesquisa. Substitui um byte em AL por um byte de uma tabela de tradução codificada pelo usuário. Quando XLAT é executada, AL deverá ter um índice sem sinal para a tabela. XLAT muda o conteúdo de AL do índice de tabela para a entrada de tabela.
IN, OUT	Operando de entrada, saída do espaço de E/S.
Aritméticas	
ADD	Adiciona operandos.
SUB	Subtrai operandos.
MUL	Multiplicação de inteiro sem sinal, com operandos de byte, palavra ou palavra dupla, e resultado de palavra, palavra dupla ou quatro palavras.
IDIV	Divisão com sinal.
Lógicas	
AND	AND dos operandos.
BTS	Teste e definição de bit. Opera sobre um operando de campo de bit. A instrução copia o valor atual de um bit para o flag CF e define o bit original como 1.
BSF	Varredura de bit adiante. Varre uma palavra ou palavra dupla para o bit 1 e armazena o número do primeiro bit 1 em um registrador.
SHL/SHR	Deslocamento lógico à esquerda ou à direita.
SAL/SAR	Deslocamento aritmético à esquerda ou à direita.
ROL/ROR	Rotação à esquerda ou à direita.
SETcc	Define um byte como zero ou 1, dependendo de qualquer uma das 16 condições definidas pelos flags de status.
Transferência de controle	
JMP	Salto incondicional.
CALL	Transfere o controle para outro local. Antes da transferência, o endereço da instrução após o CALL é colocado na pilha.
JE/JZ	Salto se igual/zero.
LOOPE/LOOPZ	Loop se igual/zero. Esse é um salto condicional usando um valor armazenado no registrador ECX. A primeira instrução decrementa ECX antes de testá-lo para a condição de desvio.
INT/INTO	Interrompe/Interrompe se houver <i>overflow</i> . Transfere o controle para uma rotina de serviço de interrupção.
Operações de string	
MOVS	Move <i>string</i> de byte, palavra, palavra dupla. A instrução opera sobre um elemento de uma <i>string</i> , indexado pelos registradores ESI e EDI. Após cada operação de <i>string</i> , os registradores são automaticamente incrementados ou decrementados para apontarem para o próximo elemento da <i>string</i> .
LDS	Carrega byte, palavra ou palavra dupla de <i>string</i> .
Suporte a linguagem de alto nível	
ENTER	Cria um frame de pilha que pode ser usado para implementar as regras de uma linguagem de alto nível estruturada em bloco.
LEAVE	Reverte a ação de um ENTER anterior.
BOUND	Verifica limites de <i>array</i> . Verifica se o valor no operando 1 está dentro dos limites inferior e superior. Os limites estão em dois locais de memória adjacentes referenciados pelo operando 2. Uma interrupção ocorre se o valor estiver fora dos limites. Essa instrução é usada para verificar um índice de <i>array</i> .
Controle de flag	
STC	Define flag de <i>carry</i> .
LAHF	Carrega registro AH a partir dos flags. Copia bits SF, ZF, AF, F e CF para o registrador A.

(Continua)

Tabela 10.8 Tipos de operação do x86 (com exemplos de operações típicas) (*continuação*)

Instrução	Descrição
Registrador de segmento	
LDS	Carrega ponteiro para DS e para outro registrador.
Controle do sistema	
HLT	Termina a execução do programa.
LOCK	Ativa uma suspensão na memória compartilhada de modo que o Pentium tenha uso exclusivo dela durante a instrução que vem imediatamente após o LOCK.
ESC	Escape para uma extensão do processador. Um código de escape que indica que as instruções seguintes devem ser executadas por um processador numérico que admite cálculos com inteiro e ponto flutuante de alta precisão.
WAIT	Espera até BUSY# negado. Suspende a execução do programa no Pentium até que o processador detecte que o pino BUSY esteja inativo, indicando que o coprocessador numérico terminou a execução.
Proteção	
SGDT	Armazena tabela de descritor global.
LSL	Carrega limite de segmento. Carrega um registrador especificado pelo usuário com um limite de segmento.
VERR/VERW	Verifica segmento para leitura/gravação.
Gerenciamento de cache	
INVD	Esvazia a memória cache interna.
WBINVD	Esvazia a memória cache interna após gravar linhas modificadas na memória.
INVLPG	Invalida uma entrada no <i>translation lookaside buffer</i> (TLB).

INSTRUÇÕES CALL/RETURN O x86 oferece quatro instruções para dar suporte à chamada ou ao retorno de procedimento: CALL, ENTER, LEAVE, RETURN. Será instrutivo examinarmos o suporte oferecido por essas instruções. Lembre-se, pela Figura 10.10, que um meio comum de implementar o mecanismo de chamada/retorno de procedimento é por meio de stack frame. Quando um novo procedimento é chamado, o seguinte deverá ser realizado na entrada do novo procedimento:

- Levar o ponto de retorno para a pilha.
- Levar o ponteiro do frames atual para a pilha.
- Copiar o ponteiro de pilha como o novo valor do ponteiro de frames.
- Ajustar o ponteiro de pilha para alocar um frames.

A instrução CALL coloca o valor do ponteiro de instrução atual na pilha e causa um salto para o ponto de entrada do procedimento, colocando o endereço do ponto de entrada no ponteiro de instrução. Nas máquinas 8088 e 8086, o procedimento típico começava com a sequência

```
PUSH    EBP
MOV     EBP, ESP
SUB     ESP, espaço_para_locais
```

onde EBP é o ponteiro de frame e ESP é o ponteiro de pilha. No 80286 e em máquinas posteriores, a instrução ENTER realiza todas as operações mencionadas em uma única instrução.

A instrução ENTER foi acrescentada ao conjunto de instruções para oferecer suporte direto para o compilador. A instrução também inclui um recurso para dar suporte aos chamados procedimentos aninhados em linguagens como Pascal, COBOL e Ada (não encontrados em C ou FORTRAN). Acontece que existem melhores maneiras de tratar de chamadas de procedimentos aninhados para essas linguagens. Além do mais, embora a instrução ENTER economize alguns bytes de memória em comparação com a sequência PUSH, MOV, SUB (4 bytes contra 6 bytes), ela na realidade leva mais tempo para executar (10 ciclos de clock contra 6 ciclos de clock). Assim, embora possa parecer uma boa ideia para os projetistas de conjunto de instruções acrescentar esse recurso, ele complica a implementação do processador, enquanto oferece pouco ou nenhum benefício. Veremos que, em comparação, uma

técnica RISC para o projeto de processador evitaria instruções complexas, como ENTER, e poderia produzir uma implementação mais eficiente com uma sequência de instruções mais simples.

GERENCIAMENTO DE MEMÓRIA Outro conjunto de instruções especializadas lida com a segmentação da memória. Estas são instruções privilegiadas que só podem ser executadas a partir do sistema operacional. Elas permitem que tabelas de segmento locais e globais (chamadas tabelas de descritores) sejam carregadas e lidas, e permitem que o nível de privilégio de um segmento seja verificado e alterado.

As instruções especiais para lidar com a cache no chip foram discutidas no Capítulo 4.

FLAGS DE STATUS E CÓDIGOS DE CONDIÇÃO Os flags de *status* são bits em registradores especiais que podem ser definidos por certas operações e usados em instruções de desvio condicional. O termo *código de condição* refere-se às configurações de um ou mais flags de *status*. No x86 e em muitas outras arquiteturas, os flags de *status* são definidos por operações aritméticas e de comparação. A operação de comparação na maioria das linguagens subtrai dois operandos, assim como uma operação de subtração. A diferença é que uma operação de comparação só define flags de *status*, enquanto uma operação de subtração também armazena o resultado da subtração no operando de destino. Algumas arquiteturas também definem os flags de *status* para instruções de transferência de dados.

A Tabela 10.9 lista os flags de *status* usados no x86. Cada flag, ou combinações de flags, podem ser testados para um salto condicional. A Tabela 10.10 mostra os códigos de condição (combinações de valores de flag de *status*) para os quais os *opcodes* de salto foram definidos.

Várias observações interessantes podem ser feitas sobre essa lista. Primeiro, podemos querer testar dois operandos para determinar se um número é maior que outro. Mas isso dependerá de os números terem sinal ou não. Por exemplo, o número de 8 bits 11111111 é maior que 00000000 se os dois números forem interpretados como inteiros sem sinal ($255 > 0$), mas é menor se eles forem considerados como números de 8 bits por complemento de dois ($-1 < 0$). Muitas linguagens *assembly*, portanto, introduzem dois conjuntos de termos para distinguir os dois casos: se estivermos comparando dois números como inteiros com sinal, usamos os termos *menor que* e *maior que*; se os estivermos comparando como inteiros sem sinal, usamos os termos *abaixo* e *acima*.

Uma segunda observação refere-se à complexidade da comparação de inteiros com sinal. Um resultado com sinal é maior ou igual a zero se (1) o bit de sinal for zero e não houver *overflow* ($S = 0 \text{ AND } O = 0$), ou (2) o bit de sinal for 1 e houver um *overflow*. Um estudo da Figura 9.4 deverá convencê-lo de que as condições testadas para as várias operações com sinal são apropriadas.

INSTRUÇÕES SIMD DO X86 Em 1996, a Intel introduziu a tecnologia MMX em sua linha de produtos Pentium. MMX é um conjunto de instruções altamente otimizado para tarefas de multimídia. Existem 57 novas instruções que tratam de dados em um padrão SIMD (única instrução, múltiplos dados), possibilitando realizar a mesma operação, como adição ou multiplicação, sobre múltiplos elementos de dados ao mesmo tempo. Cada instrução

Tabela 10.9 Flags de status do x86

Bit de status	Nome	Descrição
C	Carry	Indica a existência do bit de transporte ou empréstimo (<i>carry bit</i>) na posição do bit mais à esquerda após uma operação aritmética. Também modificado por algumas das operações de deslocamento e rotação.
P	Parity	Paridade do byte menos significativo do resultado de uma operação aritmética ou lógica. 1 indica paridade par; 0 indica paridade ímpar.
A	Auxiliary carry	Representa a existência do bit de transporte ou empréstimo (<i>carry bit</i>) na posição entre dois bytes após uma operação aritmética ou lógica de 8 bits. Usado na aritmética BCD.
Z	Zero	Indica que o resultado de uma operação aritmética ou lógica é 0.
S	Sign	Indica o sinal do resultado de uma operação aritmética ou lógica.
O	Overflow	Indica um <i>overflow</i> aritmético após uma adição ou subtração de aritmética de complemento a dois.

Tabela 10.10 Códigos de condição do x86 para instruções de salto condicional e SETcc

Símbolo	Condição testada	Comentário
A, NBE	$C = 0 \text{ AND } Z = 0$	Acima; Não abaixo ou igual (maior que, sem sinal)
AE, NB, NC	$C = 0$	Acima ou igual; Não abaixo (maior que ou igual, sem sinal); Sem carry
B, NAE, C	$C = 1$	Abaixo; Não acima ou igual (menor que, sem sinal); Carry definido
BE, NA	$C = 1 \text{ OR } Z = 1$	Abaixo ou igual; Não acima (menor que ou igual, sem sinal)
E, Z	$Z = 1$	Igual; Zero (com ou sem sinal)
G, NLE	$[(S = 1 \text{ AND } O = 1) \text{ OR } (S = 0 \text{ AND } O = 0)]$ $\text{AND } [Z = 0]$	Maior que; Não menor que ou igual (com sinal)
GE, NL	$(S = 1 \text{ AND } O = 1) \text{ OR } (S = 0 \text{ AND } O = 0)$	Maior que ou igual; Não menor que (com sinal)
L, NGE	$(S = 1 \text{ AND } O = 0) \text{ OR } (S = 0 \text{ AND } O = 1)$	Menor que; Não maior que ou igual (com sinal)
LE, NG	$(S = 1 \text{ AND } O = 0) \text{ OR } (S = 0 \text{ AND } O = 1) \text{ OR } (Z = 1)$	Menor que ou igual; Não maior que (com sinal)
NE, NZ	$Z = 0$	Não igual; Não zero (com ou sem sinal)
NO	$O = 0$	Sem overflow
NS	$S = 0$	Sem sinal (não negativo)
NP, PO	$P = 0$	Sem paridade; Paridade ímpar
O	$O = 1$	Overflow
P	$P = 1$	Paridade; Paridade par
S	$S = 1$	Sinal (negativo)

normalmente utiliza um único ciclo de clock para ser executada. Para a aplicação correta, essas operações paralelas rápidas podem gerar um ganho de velocidade de 2 a 8 vezes em comparação com algoritmos que não usam as instruções MMX (Atkins, 1996²). Com a introdução da arquitetura x86 de 64 bits, a Intel expandiu essa extensão para incluir operandos de quatro palavras (*double quadwords*) duplos (128 bits) e operações de ponto flutuante. Nesta subseção, descrevemos os recursos do MMX.

O foco do MMX é a programação multimídia. Dados de vídeo e áudio normalmente são compostos de grandes arrays de pequenos tipos de dados, como 8 ou 16 bits, enquanto as instruções convencionais são ajustadas para operar sobre dados de 32 ou 64 bits. Aqui estão alguns exemplos: em gráficos e vídeo, uma única cena consiste em um *array* de pixels,² e existem 8 bits para cada pixel ou 8 bits para cada componente de cor do pixel (vermelho, verde, azul). As amostras de áudio típicas são quantizadas usando 16 bits. Para alguns algoritmos gráficos 3D, 32 bits são comuns para os tipos de dados básicos. Para permitir operação paralela sobre esses tamanhos de dados, três novos tipos de dados são definidos em MMX. Cada tipo de dados tem 64 bits de extensão e consiste em múltiplos campos de dados menores, cada um mantendo um inteiro de ponto fixo. Os tipos são os seguintes:

- **Pacote de bytes:** oito bytes agrupados em uma quantidade de 64 bits.
- **Pacote de palavras:** quatro palavras de 16 bits agrupadas em 64 bits.
- **Pacote de palavras duplas:** duas palavras duplas de 32 bits agrupadas em 64 bits.

² Um pixel, ou elemento de imagem, é o menor elemento de uma imagem digital que pode receber um nível de cinza. De modo equivalente, um pixel é um ponto individual em uma representação de matriz de pontos de uma figura.

A Tabela 10.11 lista o conjunto de instruções MMX. A maior parte das instruções envolve a operação paralela sobre bytes, palavras ou palavras duplas. Por exemplo, a instrução PSLW realiza um deslocamento lógico à esquerda separadamente em cada uma das quatro palavras no operando de pacote de palavras; a instrução PADDB apanha os operandos de pacotes de bytes como entrada e realiza adições paralelas em cada posição de byte independentemente para produzir uma saída de pacotes de bytes.

Tabela 10.11 Conjunto de instruções MMX

Categoria	Instrução	Descrição
Aritmética	PADD [B, W, D]	Adição paralela de oito pacotes de bits, quatro palavras de 16 bits ou duas palavras duplas de 32 bits, com contorno.
	PADD [B, W]	Adição com saturação.
	PADDUS [B, W]	Adição sem sinal com saturação.
	PSUB [B, W, D]	Subtração com contorno.
	PSUBS [B, W]	Subtração com saturação.
	PSUBUS [B, W]	Subtração sem sinal com saturação.
	PMULHW	Multiplicação paralela de quatro palavras de 16 bits com sinal, com 16 bits de alta ordem do resultado de 32 bits escolhidos.
	PMULLW	Multiplicação paralela de quatro palavras de 16 bits com sinal, com 16 bits de baixa ordem do resultado de 32 bits escolhidos.
	PMADDWD	Multiplicação paralela de quatro palavras de 16 bits com sinal; soma pares adjacentes de resultados de 32 bits
Comparação	PCMPEQ [B, W, D]	Comparação paralela de igualdade; resultado é máscara de 1s se verdadeiro ou 0s se falso.
	PCMPGT [B, W, D]	Comparação paralela de maior; resultado é máscara de 1s se verdadeiro ou 0s se falso.
Conversão	PACKUSWB	Agrupa palavras em bytes com saturação sem sinal.
	PACKSS [WB, DW]	Agrupa palavras em bytes, ou palavras duplas em palavras, com saturação com sinal.
	PUNPCKH [BW, WD, DQ]	Desagrupa em paralelo (mesclagem intervalada) bytes, palavras ou palavras duplas de alta ordem do registrador MMX.
	PUNPCKL [BW, WD, DQ]	Desagrupa em paralelo (mesclagem intervalada) bytes, palavras ou palavras duplas de baixa ordem do registrador MMX.
Lógica	PAND	AND lógico bit a bit com 64 bits.
	PANDN	AND NOT lógico bit a bit com 64 bits.
	POR	OR lógico bit a bit com 64 bits.
	PXOR	XOR lógico bit a bit com 64 bits.
Deslocamento	PSLL [W, D, Q]	Deslocamento lógico paralelo à esquerda de pacotes de palavra, palavras duplas ou quatro palavras pela quantidade especificada no registrador MMX ou valor imediato.
	PSRL [W, D, Q]	Deslocamento lógico paralelo à direita de pacotes de palavra, palavras duplas ou quatro palavras agrupadas.
	PSRA [W, D]	Deslocamento aritmético paralelo à direita de pacotes de palavra, palavras duplas ou quatro palavras.
Transferência de dados	MOV [D, Q]	Move palavras duplas ou quatro palavras de/para registrador MMX.
Ger. de estado	EMMS	Esvazia estado MMX (esvazia bits de tag dos registradores FP).

Nota: se uma instrução aceitar vários tipos de dados [byte (B), palavra (W), *doubleword* (D), *quadword* (Q)], os tipos de dados são indicados entre colchetes.

Um recurso incomum do novo conjunto de instruções é a introdução da **aritmética de saturação** para operandos de byte e palavra de 16 bits. Com a aritmética sem sinal comum, quando em uma operação ocorre *overflow* (ou seja, um *carry* sai pelo bit mais significativo), o bit extra é truncado. Isso é conhecido como contorno, pois o efeito do truncamento pode ser, por exemplo, produzir um resultado de adição menor que os dois operandos da entrada. Considere a adição das duas palavras, em hexadecimal, F00h e 3000h. A soma seria expressa como

$$\begin{array}{r} \text{F000h} = 1111\ 0000\ 0000\ 0000 \\ + 3000\text{h} = 0011\ 0000\ 0000\ 0000 \\ \hline 10010\ 0000\ 0000\ 0000 = 2000\text{h} \end{array}$$

Se os dois números representassem intensidade de imagem, então o resultado da adição é tornar a combinação dos dois tons escuros mais clara. Isso normalmente não é o que foi intencionado. Com a aritmética de saturação, se a adição resultar em *overflow* ou a subtração resultar em *underflow*, o resultado é definido para o maior ou menor valor representável. Para o exemplo anterior, com a aritmética de saturação, temos

$$\begin{array}{r} \text{F000h} = 1111\ 0000\ 0000\ 0000 \\ + 3000\text{h} = 0011\ 0000\ 0000\ 0000 \\ \hline 10010\ 0000\ 0000\ 0000 \\ 1111\ 1111\ 1111\ 1111 = \text{FFFFh} \end{array}$$

Para dar uma ideia para uso de instruções MMX, examinamos um exemplo, tomado de Peleg, Wilkie e Weiser (1997^d). Uma aplicação de vídeo comum é o efeito *fade-out*, *fade-in*, em que uma cena gradualmente se dissolve em outra. Duas imagens são combinadas com uma média ponderada:

$$\text{Pixel_resultado} = \text{Pixel_A} \times \text{fade} + \text{Pixel_B} \times (1 - \text{fade})$$

Esse cálculo é realizado sobre cada posição de pixel em A e B. Se uma série de frames de vídeo for produzida enquanto se muda gradualmente o valor de *fade* de 1 para 0 (escalados devidamente para um inteiro de 8 bits), o resultado é o *fade* da imagem A para a imagem B.

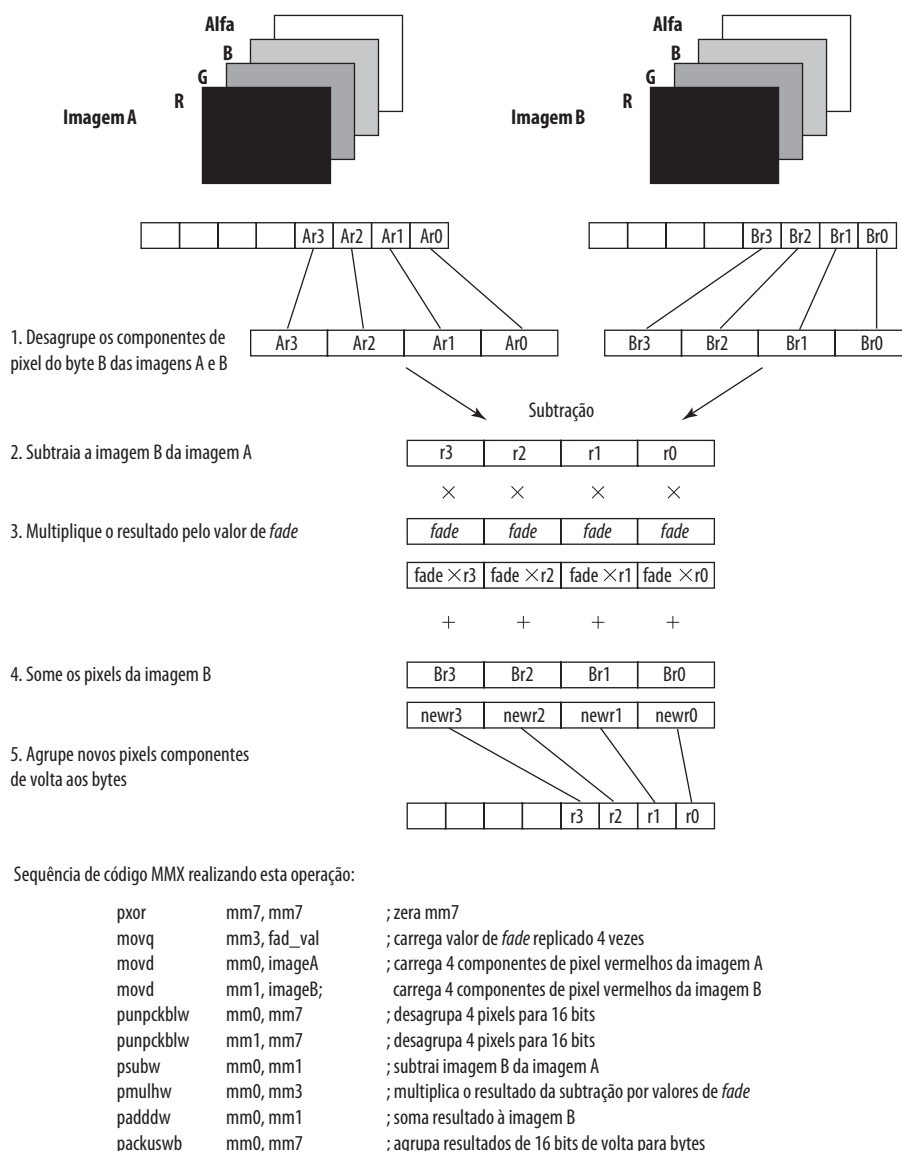
A Figura 10.11 mostra a sequência de etapas exigidas para um conjunto de pixels. Os componentes de pixel de 8 bits são convertidos para elementos de 16 bits para acomodar a capacidade de multiplicação em 16 bits do MMX. Se essas imagens utilizarem resolução de 640 × 480, e a técnica de dissolução usar todos os 255 valores possíveis do valor de *fade*, então o número total de instruções executadas usando MMX é 535 milhões. O mesmo cálculo, realizado sem as instruções MMX, requer 1,4 bilhão de execuções de instruções (Intel, 1998^e).



Tipos de operação do ARM

A arquitetura ARM oferece uma grande variedade de tipos de operando. A seguir estão as principais categorias:

- **Instruções *load e store*:** na arquitetura ARM, somente instruções *load* e *store* acessam locais da memória; instruções aritméticas e lógicas são realizadas apenas sobre registradores e os valores imediatos são codificados na instrução. Essa limitação é característica do projeto RISC e é explorada com mais detalhes no Capítulo 13. A arquitetura ARM admite dois tipos gerais de instruções que carregam ou armazenam o valor de um único registrador, ou um par de registradores, de ou para a memória: (1) carregar ou armazenar uma palavra de 32 bits ou um byte sem sinal de 8 bits, e (2) carregar ou armazenar uma meia-palavra sem sinal de 16 bits, e carregar e estender o sinal de uma meia-palavra de 16 bits ou um byte de 8 bits.
- **Instruções de desvio:** o ARM admite uma instrução de desvio que permite um desvio condicional para frente ou para trás em até 32 MB. Como o contador de programa é um dos registradores de uso geral (R15), um desvio ou salto também pode ser gerado escrevendo um valor em R15. Uma chamada de sub-rotina pode ser realizada por uma variante da instrução de desvio padrão. Além de permitir um desvio para frente ou para trás em até 32 MB, a instrução *Branch with Link* (BL) guarda o endereço da instrução após o desvio (o endereço de retorno) no LR (R14). Os desvios são determinados por um campo de condição de 4 bits na instrução.
- **Instruções de processamento de dados:** essa categoria inclui instruções lógicas (AND, OR, XOR), instruções de adição e subtração, e instruções de teste e comparação.
- **Instruções de multiplicação:** as instruções de multiplicação de inteiros operam sobre operandos de palavra ou meia-palavra e podem produzir resultados normais ou grandes. Por exemplo, existe uma instrução de multiplicação que apanha dois operandos de 32 bits e produz um resultado de 64 bits.
- **Instruções paralelas de adição e subtração:** além das instruções normais de processamento de dados e multiplicação, existe um conjunto de instruções paralelas de adição e subtração, em que partes dos dois

Figura 10.11 Composição de imagem na representação do plano de cores

operandos são operadas em paralelos. Por exemplo, ADD16 soma as meias-palavras do topo dos dois registradores para formar a meia-palavra superior do resultado e soma as meias-palavras inferiores dos mesmos dois registradores para formar a meia-palavra inferior do resultado. Essas instruções são úteis em aplicações de processamento de imagem, semelhantes às instruções MMX do x86.

- **Instruções de extensão:** existem várias instruções para desagrupar dados, estendendo por sinal ou com zeros, de bytes para meias-palavras ou palavras, ou de meias-palavras para palavras.
- **Instruções de acesso do registrador de *status*:** o ARM oferece a capacidade de ler e também escrever em partes do registrador de *status*.

CÓDIGOS DE CONDIÇÃO A arquitetura ARM define quatro flags de condição que são armazenados no registrador de *status* do programa: N, Z, C e V (Negativo, Zero, Carry e Overflow), com significados basicamente iguais aos flags S, Z, C e V na arquitetura x86. Esses quatro flags constituem um código de condição no ARM. A Tabela 10.12 mostra a combinação de condições para as quais a execução condicional é definida.

Tabela 10.12 Condições do ARM para execução de instrução condicional

Código	Símbolo	Condição Testada	Comentário
0000	EQ	$Z = 1$	Igual
0001	NE	$Z = 0$	Não igual
0010	CS/HS	$C = 1$	Carry 'setado'/acima ou igual sem 'sinal'
0011	CC/LO	$C = 0$	Carry zerado/abaixo sem sinal
0100	MI	$N = 1$	Menos/negativo
0101	PL	$N = 0$	Mais/positivo ou zero
0110	VS	$V = 1$	Overflow
0111	VC	$V = 0$	Nenhum overflow
1000	HI	$C = 1 \text{ AND } Z = 0$	Acima sem sinal
1001	LS	$C = 0 \text{ OR } Z = 1$	Abaixo ou igual sem sinal
1010	GE	$N = V$ $[(N = 1 \text{ AND } V = 1) \text{ OR } (N = 0 \text{ AND } V = 0)]$	Sinalizado maior que ou igual
1011	LT	$N \neq V$ $[(N = 1 \text{ AND } V = 0) \text{ OR } (N = 0 \text{ AND } V = 1)]$	Sinalizado menor que
1100	GT	$(Z = 0) \text{ AND } (N = V)$	Sinalizado maior que
1101	LE	$(Z = 1) \text{ OR } (N \neq V)$	Sinalizado menor que ou igual
1110	AL	—	Sempre (incondicional)
1111	—	—	Esta instrução só pode ser executada incondicionalmente

Existem dois aspectos incomuns no uso dos códigos de condição no ARM:

1. Todas as instruções, não apenas as de desvio, incluem um campo de código de condição, o que significa que praticamente todas as instruções podem ser executadas condicionalmente. Qualquer combinação de valores de flag, exceto 1110 ou 1111 no campo de código de condição de uma instrução significa que a instrução só será executada se a condição for atendida.
2. Todas as instruções de processamento de dados (aritméticas, lógicas) incluem um bit S, que indica se a instrução atualiza os flags de condição.

O uso da execução condicional e de valores condicionais dos flags de condição ajuda no projeto de programas mais curtos, que usam menos memória. Por outro lado, todas as instruções incluem 4 bits para o código de condição, de modo que existe uma escolha, porque menos bits na instrução de 32 bits estão disponíveis para o *opcode* e os operandos. Como o ARM é um projeto RISC que conta bastante com o endereçamento do registrador, essa parece ser uma escolha razoável.



10.6 Leitura recomendada

O conjunto de instruções do x86 é muito bem explicado por Brey (2009^f). O conjunto de instruções do ARM é explicado em Sloss, Symes e Wright (2004^g) e Knaggs e Welsh (2004^h). Intel (2004ⁱ) descreve considerações de software relacionadas à arquitetura *endian* do microprocessador, e discute diretrizes para o desenvolvimento de código independente do tipo *endian*.

Principais termos, perguntas de revisão e problemas

Principais termos

Acumulador	Salto	Chamada de procedimento
Endereço	<i>Little-endian</i>	Retorno de procedimento
Deslocamento aritmético	Deslocamento lógico	Push
<i>Bi-endian</i>	Instrução de máquina	Procedimento reentrante
<i>Big-endian</i>	Operando	Notação polonesa invertida
Desvio	Operação	Rotação
Desvio condicional	Decimal agrupado	Salto
Conjunto de instruções	Pop	Pilha

Perguntas de revisão

- 10.1 Quais são os elementos típicos de uma instrução de máquina?
- 10.2 Que tipos de locais podem manter operandos de origem e destino?
- 10.3 Se uma instrução contém quatro endereços, qual poderia ser a finalidade de cada endereço?
- 10.4 Liste e explique resumidamente cinco questões importantes no projeto do conjunto de instruções.
- 10.5 Que tipos de operandos são típicos nos conjuntos de instrução de máquina?
- 10.6 Qual é o relacionamento entre o código de caracteres IRA e a representação decimal agrupada?
- 10.7 Qual é a diferença entre um deslocamento aritmético e um deslocamento lógico?
- 10.8 Por que são necessárias instruções de transferência de controle?
- 10.9 Liste e explique resumidamente duas maneiras comuns de gerar a condição a ser testada em uma instrução de desvio condicional.
- 10.10 O que significa o termo *aninhamento de procedimentos*?
- 10.11 Liste três locais possíveis para armazenar o endereço de retorno para um retorno de procedimento.
- 10.12 O que é um procedimento reentrante?
- 10.13 O que é notação polonesa invertida?
- 10.14 Qual é a diferença entre *big-endian* e *little-endian*?

Problemas

- 10.1 Mostre em notação hexa:
 - a. O formato decimal agrupado para 23
 - b. Os caracteres ASCII 23
- 10.2 Para cada um dos seguintes números decimais agrupados, mostre o valor decimal:
 - a. 0111 0011 0000 1001
 - b. 0101 1000 0010
 - c. 0100 1010 0110
- 10.3 Determinado microprocessador tem palavras de 1 byte. Qual é o menor e o maior inteiro que pode ser representado nas seguintes representações:
 - a. Sem sinal
 - b. Sinal-magnitude
 - c. Complemento a um
 - d. Complemento a dois
 - e. Decimal agrupado sem sinal
 - f. Decimal agrupado com sinal

- 10.4** Muitos processadores oferecem uma lógica para realizar aritmética sobre números decimais agrupados. Embora as regras para a aritmética decimal sejam semelhantes àsquelas para operações binárias, os resultados decimais podem exigir algumas correções aos dígitos individuais se a lógica binária for usada.

Considere a adição decimal de dois números sem sinal. Se cada número consistir em N dígitos, então existem $4N$ bits em cada número. Os dois números devem ser somados usando um somador binário. Sugira uma regra simples para corrigir o resultado. Realize a adição dessa forma sobre os números 1698 e 1786.

- 10.5** O complemento de dez do número decimal X é definido como sendo $10^N - X$, onde N é a quantidade de dígitos decimais no número. Descreva o uso da representação em complemento de dez para realizar subtração decimal. Ilustre o procedimento subtraindo $(0326)_{10}$ de $(0736)_{10}$.

- 10.6** Compare máquinas de zero, um, dois e três endereços escrevendo programas para calcular

$$X = (A + B \times C) / (D - E \times F)$$

para cada uma das quatro máquinas. As instruções disponíveis para uso são as seguintes:

0 endereço	1 endereço	2 endereços	3 endereços
PUSH M	LOAD M	MOVE ($X \leftarrow Y$)	MOVE ($X \leftarrow Y$)
POP M	STORE M	ADD ($X \leftarrow X + Y$)	ADD ($X \leftarrow Y + Z$)
ADD	ADD M	SUB ($X \leftarrow X - Y$)	SUB ($X \leftarrow Y - Z$)
SUB	SUB M	MUL ($X \leftarrow X \times Y$)	MUL ($X \leftarrow Y \times Z$)
MUL	MUL M	DIV ($X \leftarrow X/Y$)	DIV ($X \leftarrow Y/Z$)
DIV	DIV M		

- 10.7** Considere um computador hipotético com um conjunto de instruções de apenas duas instruções de n bits. O primeiro bit especifica o *opcode*, e os bits restantes especificam uma das 2^{n-1} palavras de n bits da memória principal. As duas instruções são as seguintes:

SUBS X Subtrai o conteúdo do local X do acumulador e armazena o resultado no local X e no acumulador.

JUMP X Coloca o endereço X no contador de programa.

Uma palavra na memória principal pode conter uma instrução ou um número binário na notação de complemento a dois. Demonstre que esse repertório de instruções é razoavelmente completo, especificando como as seguintes operações podem ser programadas:

- Transferência de dados: local X para acumulador, acumulador para local X.
- Adição: adicionar conteúdo do local X para acumulador.
- Desvio condicional.
- OR lógico.
- Operações de E/S.

- 10.8** Muitos conjuntos de instruções contêm a instrução NOOP, significando nenhuma operação, a qual não tem efeito sobre o estado do processador, além de incrementar o contador de programa. Sugira alguns usos dessa instrução.

- 10.9** Na Seção 10.4, afirmamos que um deslocamento aritmético à esquerda e um deslocamento lógico à esquerda correspondem a uma multiplicação por 2 quando não existe *overflow*, e se houver *overflow*, as operações de deslocamento aritmético e lógico à esquerda produzem resultados diferentes, mas o deslocamento aritmético à esquerda retém o sinal do número. Demonstre que essas afirmações são verdadeiras para inteiros em complemento de dois com 5 bits.

- 10.10** De que maneira os números são arredondados usando o deslocamento aritmético à direita (por exemplo, arredondar para $+q$, arredondar para $-q$, para zero, para longe de 0)?

- 10.11** Suponha que uma pilha deva ser usada pelo processador para gerenciar chamadas e retornos de procedimento. O contador de programa pode ser eliminado usando o topo da pilha como um contador de programa?

- 10.12** A arquitetura x86 inclui uma instrução chamada *Decimal Adjust after Addition* (DAA). DAA realiza a seguinte sequência de instruções:

```

if ((AL AND 0FH) > 9) OR (AF = 1) then
    AL ← AL + 6;
    AF ← 1;
else
    AF ← 0;
endif;
if (AL > 9FH) OR (CF = 1) then
    AL ← AL + 60H;
    CF ← 1;
else
    CF ← 0;
endif.

```

“H” indica hexadecimal. AL é um registrador de 8 bits que mantém o resultado da adição de dois inteiros sem sinal com 8 bits. AF é um flag que é 'setado' se houver um *carry* do bit 3 ao bit 4 no resultado de uma adição. CF é um flag 'setado' se houver um *carry* do bit 7 ao bit 8. Explique a função realizada pela instrução DAA.

- 10.13** A instrução x86 Compare (CMP) subtrai o operando de origem do operando de destino; ela atualiza os flags de *status* (C, P, A, Z, S, O), mas não altera operando algum. A instrução CMP pode ser usada para determinar se o operando de destino é maior, igual ou menor que o operando de origem.
- Suponha que os dois operandos sejam tratados como inteiros sem sinal. Mostre quais flags de *status* são relevantes para determinar o tamanho relativo dos dois inteiros e que valores dos flags correspondem a maior, igual ou menor.
 - Suponha que os dois operandos sejam tratados como inteiros com sinal em complemento de dois. Mostre quais flags de *status* são relevantes para determinar o tamanho relativo dos dois inteiros e que valores dos flags correspondem a maior, igual ou menor.
 - A instrução CMP pode ser seguida por uma instrução *Jump condicional* (Jcc) ou *Set Condition* (SETcc), onde cc refere-se a uma das 16 condições listadas na Tabela 10.10. Demonstre que as condições testadas para uma comparação numérica com sinal são corretas.
- 10.14** Suponha que queiramos aplicar a instrução CMP do x86 aos operandos de 32 bits que contenham números em um formato de ponto flutuante. Para obter resultados corretos, que requisitos precisam ser atendidos nas áreas a seguir?
- A posição relativa dos campos de significando, sinal e expoente.
 - A representação do valor zero.
 - A representação do expoente.
 - O formato do IEEE cumpre esses requisitos? Explique.
- 10.15** Muitos conjuntos de instrução de microprocessador incluem uma instrução que testa uma condição e define um operando de destino se a condição for verdadeira. Alguns exemplos incluem o SETcc no x86, o Scc no Motorola MC68000 e o Scond no National NS32000.
- Existem algumas diferenças entre essas instruções:
 - SETcc e Scc operam apenas sobre um byte, enquanto Scond opera sobre operandos de byte, palavra e palavra dupla.
 - SETcc e Scond definem o operando como o inteiro 1 se verdadeiro e zero se falso. Scc define o byte com 1s binários se verdadeiro e 0s se falso.
 Quais são as vantagens e desvantagens relativas dessas diferenças?
 - Nenhuma dessas instruções define qualquer um dos flags de código de condição, e assim é preciso haver um teste explícito do resultado da instrução para determinar seu valor. Discuta se os códigos de condição devem ser definidos como um resultado dessa instrução.
 - Uma instrução IF simples como IF $a > b$ THEN pode ser implementada com um método de representação numérica, ou seja, tornando o valor booleano explícito, ao contrário de um método de *fluxo de controle*, que representa o valor de uma expressão booleana por um certo ponto no programa. Um compilador poderia implementar IF $a > b$ THEN com o seguinte código x86:

SUB	CX, CX	; atualizar CX com 0
MOV	AX, B	; mover o conteúdo da localização de memória B para o registrador AX
CMP	AX, A	; comparar o conteúdo do registrador AX e da localização de memória A
JLE	TEST	; salta se $A \leq B$
INC	CX	; somar 1 ao conteúdo do registrador CX
TEST	JCXZ	OUT
THEN	OUT	; salta se o conteúdo de CX é igual a 0

O resultado de $(A > B)$ é um valor booleano mantido em um registrador e disponível depois, fora do contexto do fluxo de código mostrado. É conveniente usar o registrador CX para isso, pois muitos dos *opcodes* de desvio e loop possuem um teste embutido para CX.

Mostre uma implementação alternativa usando a instrução SETcc que economize memória e tempo de execução. (Dica: nenhuma instrução x86 nova é necessária, além de SETcc.)

d. Agora considere a instrução em linguagem de alto nível:

$A := (B > C) \text{ OR } (D = F)$

Um compilador poderia gerar o seguinte código:

	MOV	EAX, B	; move conteúdo da localização B para registrador EAX
	CMP	EAX, C	; compara conteúdo do registrador EAX e da localização C
	MOV	BL, 0	; 0 representa falso
	JLE	N1	; salta se $B \leq C$
	MOV	BL, 1	; 1 representa falso
N1	MOV	EAX, D	
	CMP	EAX, F	
	MOV	BH, 0	
	JNE	N2	
	MOV	BH, 1	
N2	OR	BL, BH	

Mostre uma implementação alternativa usando a instrução SETcc que economize memória e tempo de execução.

10.16 Suponha que dois registradores contêm os seguintes valores hexadecimais: AB0890C2, 4598EE50. Qual é o resultado de somá-los usando instruções MMX?

- Para pacote de bytes
- Para pacote de palavras

Considere que a aritmética de saturação não é utilizada.

10.17 O Apêndice 10A indica que não existem instruções orientadas a pilha em um conjunto de instruções se a pilha tiver que ser usada apenas pelo processador para finalidades tais como tratamento de procedimento. Como o processador pode usar uma pilha para qualquer finalidade sem instruções orientada a pilha?

10.18 Converta as seguintes fórmulas da notação polonesa invertida para infix:

- $AB + C + D \times$
- $AB/CD / +$
- $ABCDE + \times \times /$
- $ABCDE + F / + G - H / \times +$

10.19 Converta as seguintes fórmulas da notação infix para polonesa invertida:

- $A + B + C + D + E$
- $(A + B) \times (C + D) + E$
- $(A \times B) + (C \times D) + E$
- $(A - B) \times (((C - D \times E)/F)/G) \times H$

10.20 Converta a expressão $A + B - C$ para a notação de pós-fixado usando o algoritmo de Dijkstra. Mostre as etapas envolvidas. O resultado é equivalente a $(A + B) - C$ ou $A + (B - C)$? Isso faz diferença?

10.21 Usando o algoritmo para converter infix para pós-fixado, definido no Apêndice 10A, mostre as etapas envolvidas na conversão da expressão da Figura 10.15 para pós-fixado. Use uma apresentação semelhante à da Figura 10.17.

10.22 Mostre o cálculo da expressão na Figura 10.17, usando uma apresentação semelhante à da Figura 10.16.

10.23 Redesenhe o layout *little-endian* da Figura 10.18 de modo que os bytes apareçam como numerados no layout *big-endian*. Ou seja, mostre a memória em linhas de 64 bits, com os bytes listados da esquerda para a direita, de cima para baixo.

10.24 Para as seguintes estruturas de dados, desenhe os layouts *big-endian* e *little-endian*, usando o formato da Figura 10.18, e comente os resultados.

```

a. struct {
    double i;    //0x1112131415161718
} s1;
b. struct {
    int i;       //0x11121314
    int j;       //0x15161718
} s2;
c. struct {
    short i;     //0x1112
    short j;     //0x1314
    short k;     //0x1516
    short l;     //0x1718
} s3;

```

10.25 A especificação de arquitetura IBM Power não dita como um processador deve implementar o modo *little-endian*. Ela especifica apenas a visão da memória que um processador precisa ter ao operar no modo *little-endian*. Ao converter uma estrutura de dados de *big-endian* para *little-endian*, os processadores são livres para implementar um mecanismo de troca de byte ou usar algum tipo de mecanismo de modificação de endereço. Os processadores Power atuais são todos máquinas *big-endian* por padrão e usam a modificação de endereço para tratar dados como *little-endian*.

Considere a estrutura *s* definida na Figura 10.18. O layout na parte inferior direita da figura mostra a estrutura *s* conforme vista pelo processador. Na verdade, se a estrutura *s* for compilada no modo *little-endian*, seu layout na memória aparece na Figura 10.12. Explique o mapeamento envolvido, descreva um modo fácil de implementar o mapeamento e discuta a eficácia dessa técnica.

10.26 Escreva um pequeno programa para determinar o tipo de *endian* da máquina e informar os resultados. Execute o programa em qualquer computador que lhe esteja disponível e informe a saída.

10.27 O processador MIPS pode ser definido para operar no modo *big-endian* ou *little-endian*. Considere a instrução *Load Byte unsigned* (LBU), que carrega um byte da memória para os 8 bits de baixa ordem de um registrador e preenche os 24 bits de alta ordem do registrador com zeros. A descrição de LBU é dada no manual de referência do MIPS, usando uma linguagem de transferência de registrador como

```

mem ← LoadMemory(...)
byte ← VirtualAddress1..0
if CONDITION then
    GPR[rt] ← 024 || mem31-8 × byte .. 24-8 × byte
else
    GPR[rt] ← 024 || mem7+8 × byte .. 8 × byte
endif

```

onde *byte* refere-se aos dois bits de baixa ordem do endereço efetivo e *mem* refere-se ao valor carregado da memória. No manual, em vez da palavra *CONDITION*, uma das duas palavras a seguir é usada: *BigEndian*, *LittleEndian*. Qual palavra é usada?

Figura 10.12 Estrutura *s* *little-endian* da arquitetura Power na memória

Endereço de byte	Mapeamento de endereço <i>little-endian</i>															
	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00	00	01	02	03	11	12	13	14	21	22	23	24	25	26	27	28
08	08	09	0A	0B	0C	0D	0E	0F	D	C	B	A	31	32	33	34
10	10	11	12	13	14	15	16	17								
18	18	19	1A	1B	1C	1D	1E	1F	51	52			G	F	E	
20	20	21	22	23	24	25	26	27	61	62	63	64				

- 10.28** A maior parte, mas não todos os processadores, utiliza a ordenação de bits *big-endian* ou *little-endian* dentro de um byte que é coerente com a ordenação *big* ou *little-endian* dos bytes dentro de um escalar multibytes. Vamos considerar o Motorola 68030, que usa a ordenação de bytes *big-endian*. A documentação do 68030 referente a formatos é confusa. O manual do usuário explica que a ordenação de bit dos campos de bit é o oposto da ordenação de bit dos inteiros. A maior parte das operações com campo de bit opera com uma ordenação *endian*, mas algumas operações com campo de bit exigem a ordenação oposta. A descrição a seguir, do manual do usuário, ilustra a maior parte das operações de campo de bit:
- Um operando de bit é especificado por um endereço de base que seleciona um byte na memória (o byte de base), e um número de bit que seleciona um bit nesse byte. O bit mais significativo é o bit sete. O operando de campo de bit é especificado por: **(1)** um endereço de base que seleciona um byte na memória; **(2)** um deslocamento de campo de bit que indica o bit mais à esquerda (base) do campo de bit em relação ao bit mais significativo do byte de base; e **(3)** uma largura de campo de bit que determina quantos bits à direita do byte de base estão no campo de bit. O bit mais significativo do byte de base é o deslocamento do campo de bit 0, o bit menos significativo do byte de base é o deslocamento do campo de bit 7. Essas instruções utilizam a ordenação de bits *big-endian* ou *little-endian*?



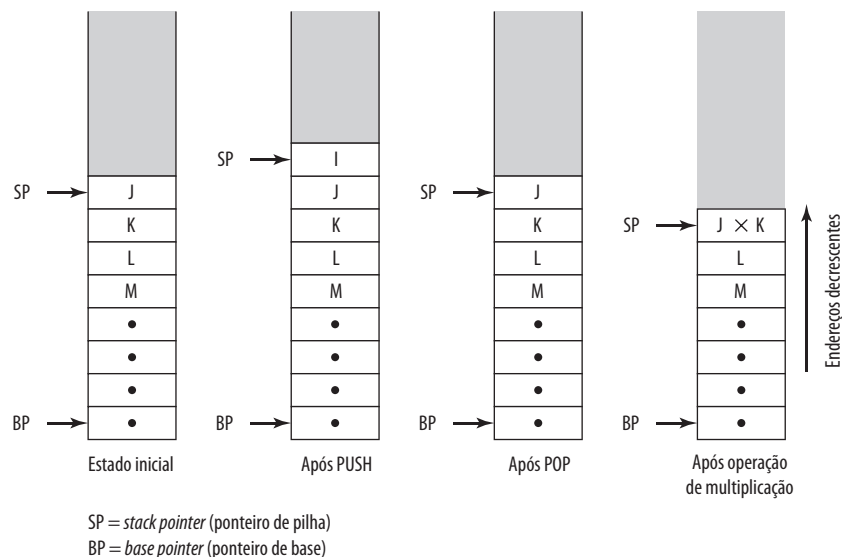
Apêndice 10A Pilhas

Pilhas

Uma pilha é um conjunto ordenado de elementos dos quais somente um pode ser acessado de cada vez. O ponto de acesso é chamado de *topo* da pilha. O número de elementos na pilha, ou *tamanho* da pilha, é variável. O último elemento na pilha é a *base* da pilha. Os itens só podem ser acrescentados ou excluídos do topo da pilha. Por esse motivo, uma pilha também é conhecida como *lista pushdown*³ ou *lista último-a-entrar-primeiro-a-sair* (ou *LIFO*—*Last In First Out*).

A Figura 10.13 mostra as operações básicas da pilha. Começamos em algum ponto no tempo em que a pilha contém alguma quantidade de elementos. Uma operação *PUSH* acrescenta um novo item ao topo da pilha. Uma operação *POP* remove o item do topo da pilha. Nos dois casos, o topo da pilha se move conforme a operação. Os operadores binários, que exigem dois operandos (por exemplo, multiplicar, dividir, somar, subtrair), utilizam os dois itens do topo da pilha como operandos, removem os dois itens e colocam o resultado de volta à pilha. As operações unárias, que exigem apenas um operando (por exemplo, NOT lógico), utilizam o item no topo da pilha. Todas essas operações são resumidas na Tabela 10.13.

Figura 10.13 Operação básica da pilha (cheia/decrecente)



³ Um termo melhor seria *colocar-no-topo da lista* (*place-on-top-of-list*), pois os elementos existentes da lista não são movidos para a memória, mas um novo elemento é acrescentado no próximo endereço de memória disponível.

Tabela 10.13 Operações orientadas à pilha

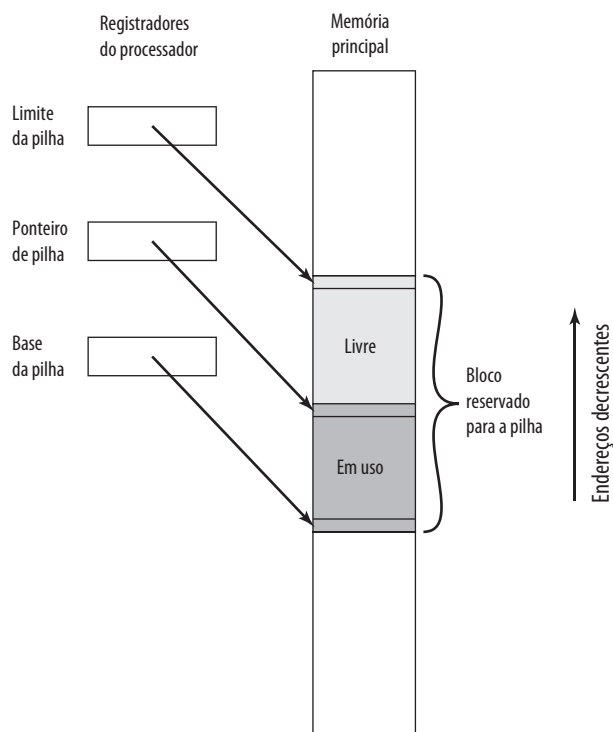
PUSH	Acrescenta um novo elemento ao topo da pilha.
POP	Retira o elemento do topo da pilha.
Operação unária	Realiza operação sobre o elemento do topo da pilha. Substitui o elemento do topo pelo resultado.
Operação binária	Realiza operação sobre os dois elementos do topo da pilha. Exclui os dois elementos da pilha. Coloca o resultado da operação no topo da pilha.

Implementação da pilha

A pilha é uma estrutura útil para ser fornecida como parte da implementação de um processador. Um de seus usos, já discutido na Seção 10.4, é gerenciar as chamadas e os retornos de procedimento. As pilhas também podem ser úteis ao programador. Um exemplo disso é a avaliação de expressões, discutida mais adiante nesta seção.

A implementação de uma pilha depende em parte dos seus usos em potencial. Se for desejado tornar as operações da pilha disponíveis ao programador, então o conjunto de instruções incluirá operações orientadas a pilha, como PUSH, POP e operações que usam um ou dois elementos do topo da pilha como operandos. Como todas essas operações se referem a um local exclusivo, a saber, o topo da pilha, o endereço do operando ou operandos é implícito e não precisa ser incluído na instrução. Estas são as instruções de zero endereços, a que nos referimos na Seção 10.1.

Se o mecanismo de pilha tiver que ser usado apenas pelo processador, para fins como tratamento de procedimento, então não haverá instruções explícitas orientadas a pilha no conjunto de instruções. De qualquer forma, a implementação de uma pilha requer que haja algum conjunto de locais usados para armazenar os elementos da pilha. Uma técnica típica é ilustrada na Figura 10.14. Um bloco contíguo de locais é reservado na memória principal (ou memória virtual) para a pilha. Quase sempre, o bloco é parcialmente preenchido com os elementos da pilha e o restante está disponível para crescimento da pilha.

Figura 10.14 Típica organização da pilha (completa/decrescente)

Três endereços são necessários para a operação correta, e estes normalmente são armazenados nos registradores do processador:

- **Ponteiro de pilha (SP, do inglês *stack pointer*):** Contém o endereço do topo da pilha. Se um item for acrescentado ou removido da pilha, o ponteiro é incrementado ou decrementado para conter o endereço do novo topo da pilha.
- **Base da pilha:** contém o endereço do local inferior do bloco reservado. Se for feita uma tentativa de POP quando a pilha estiver vazia, um erro é informado.
- **Limite da pilha:** contém o endereço da outra extremidade do bloco reservado. Se for feita uma tentativa de PUSH quando o bloco estiver totalmente utilizado para a pilha, um erro é informado.

As implementações de pilha possuem dois atributos-chave:

- **Crescente/decrecente:** uma pilha crescente cresce na direção dos endereços maiores, começando de um endereço baixo e prosseguindo para um endereço mais alto. Ou seja, uma pilha crescente é aquela em que o SP é incrementado quando os itens são acrescentados e decrementado quando os itens são removidos. Uma pilha decrescente cresce na direção dos endereços menores, começando de um endereço alto e prosseguindo para um endereço mais baixo. A maioria das máquinas implementa pilhas decrescentes como padrão.
- **Cheio/vazio:** essa é uma terminologia confusa, pois não se refere a se a pilha está completamente cheia ou completamente vazia. Em vez disso, o SP pode apontar para o item do topo na pilha (método cheio) ou para o próximo espaço livre na pilha (método vazio). Para o método cheio, quando a pilha está completamente cheia, o SP aponta para o limite superior da pilha. Para o método vazio, quando a pilha está completamente vazia, o SP aponta para a base da pilha.

A Figura 10.13 é um exemplo de uma implementação decrescente/cheia (supondo que os endereços numericamente inferiores sejam representados mais no alto da página). A arquitetura ARM permite que o programador de sistemas especifique o uso de operações de pilha crescente ou decrescente, vazia ou cheia. A arquitetura x86 utiliza uma convenção decrescente/vazia.

Avaliação de expressão

As fórmulas matemáticas normalmente são expressas no que é conhecido como notação de *infixo*. Nessa forma, um operador binário aparece entre os operandos (por exemplo, $a + b$). Para expressões complexas, parênteses são usados para determinar a ordem de avaliação das expressões. Por exemplo, $a + (b \times c)$ gerará um resultado diferente de $(a + b) \times c$. Para diminuir o uso de parênteses, as operações possuem uma precedência implícita. Geralmente, a multiplicação tem precedência sobre a adição, de modo que $a + b \times c$ é equivalente a $a + (b \times c)$.

Uma técnica alternativa é conhecida como notação *polonesa invertida*, ou *pós-fixa*. Nessa notação, o operador vem após seus dois operandos. Por exemplo,

$a + b$	torna-se $a \ b \ +$
$a + (b \times c)$	torna-se $a \ b \ c \ \times \ +$
$(a + b) \times c$	torna-se $a \ b \ + \ c \ \times$

Observe que, independentemente da complexidade de uma expressão, nenhum parêntese é exigido quando se usa a notação polonesa invertida.

A vantagem da notação de pós-fixa é que uma expressão nessa forma é facilmente avaliada usando uma pilha. Uma expressão em notação de pós-fixa é varrida da esquerda para a direita. Para cada elemento da expressão, as seguintes regras são aplicadas:

1. Se o elemento for uma variável ou constante, coloque-a na pilha.
2. Se o elemento for um operador, remova os dois itens do topo da pilha, realize a operação e coloque o resultado no topo da pilha.

Após a expressão inteira ter sido varrida, o resultado está no topo da pilha.

A simplicidade desse algoritmo torna-o conveniente para avaliar expressões. Consequentemente, muitos compiladores apanharão uma expressão em uma linguagem de alto nível, a converterão para a notação de pós-fixa e depois gerarão as instruções de máquina a partir da notação. A Figura 10.15 mostra a sequência de instruções de máquina para avaliar $f = (a - b)/(c + d \times e)$ usando instruções orientadas a pilha. A figura também mostra o uso de instruções de um endereço e dois endereços. Observe que, embora as regras orientadas a pilha não fossem usadas nos dois últimos casos, a notação de pós-fixa serviu como um guia para gerar as instruções de máquina. A sequência de eventos para o programa na pilha aparece na Figura 10.16.

O processo de converter uma expressão de infix para uma expressão de pós-fixa por si só é facilmente realizado usando uma pilha. O algoritmo a seguir é atribuído a Dijkstra (1963). A expressão de infix é verificada da esquerda para a direita e a expressão de pós-fixa é desenvolvida e gerada durante a verificação. As etapas são as seguintes:

1. Examine o próximo elemento na entrada.
2. Se for um operando, mande-o para a saída.
3. Se for um parêntese inicial, coloque-o na pilha.
4. Se for um operador, então

Figura 10.15 Comparação de três programas para calcular $f = \frac{a - b}{c + (d \times e)}$

	Pilha	Registradores gerais	Registrador único
	Push a Push b Subtract Push c Push d Push e Multiply Add Divide Pop f	Load R1, a Subtract R1, b Load R2, d Multiply R2, e Add R2, c Divide R1, R2 Store R1, f	Load d Multiply e Add c Store f Load a Subtract b Divide f Store f
Número de instruções	10	7	8
Acesso à memória	10 op + 6 d	7 op + 6 d	8 op + 8 d

- Se o topo da pilha for um parêntese inicial, então coloque o operador no topo da pilha.
 - Se tiver uma prioridade mais alta que o topo da pilha (multiplicação e divisão têm maior prioridade do que adição e subtração), então coloque o operador na pilha.
 - Se não, remova a operação da pilha e mande-a para a saída, e repita a etapa 4.
5. Se for um parêntese final, remova os operadores para a saída até que um parêntese inicial seja encontrado. Remova e descarte o parêntese inicial.
 6. Se houver mais entrada, vá para a etapa 1.
 7. Se não houver mais entrada, desempilhe os operandos restantes.
- A Figura 10.17 ilustra o uso desse algoritmo. Este exemplo deverá dar ao leitor alguma ideia do poder dos algoritmos baseados em pilha.

Figura 10.16 Uso da pilha para calcular $f = (a - b) / [(d \times e) + c]$

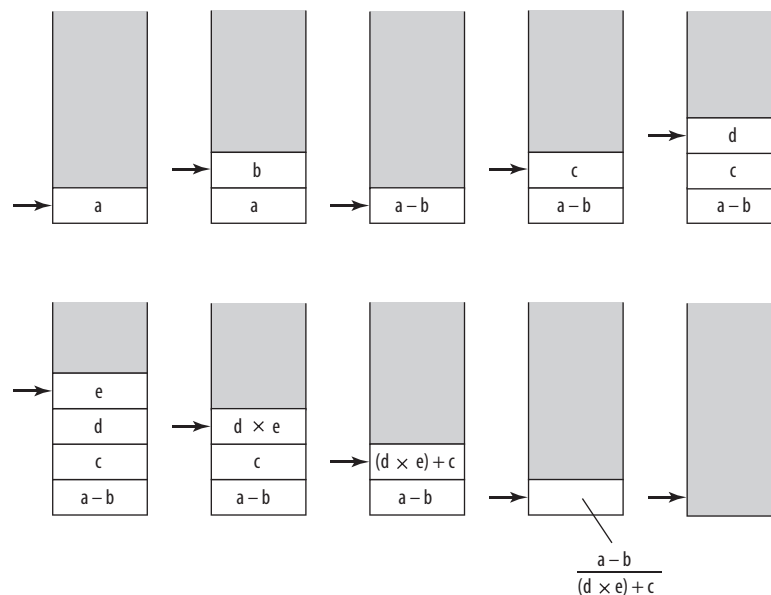


Figura 10.17 Conversão de uma expressão da notação de infix para pós-fix

Entrada	Saída	Pilha (topo à direita)
A + B × C + (D + E) × F	vazio	vazio
+ B × C + (D + E) × F	A	vazio
B × C + (D + E) × F	A	+
× C + (D + E) × F	A B	+
C + (D + E) × F	A B	+ ×
+ (D + E) × F	A B C	+ ×
(D + E) × F	A B C × +	+
D + E) × F	A B C × +	+(
+ E) × F	A B C × + D	+(
E) × F	A B C × + D	+(+
) × F	A B C × + D E	+(+
× F	A B C × + D E +	+
F	A B C × + D E +	+ ×
vazio	A B C × + D E + F	+ ×
vazio	A B C × + D E + F ×	vazio



Apêndice 10B *Little, big e bi-endian*

Um fenômeno incômodo e curioso se relaciona como os bytes dentro de uma palavra e os bits dentro de um byte são referenciados e representados. Primeiro, examinamos o problema da ordenação de bytes e depois consideramos a dos bits.

Ordenação de byte

O conceito do tipo de *endian* foi discutido inicialmente na literatura por Cohen (1981⁴). Com relação aos bytes, o tipo de *endian* tem a ver com a ordenação dos bytes de valores escalares em múltiplos bytes. A questão é mais bem entendida com um exemplo. Suponha que tenhamos o valor hexadecimal de 32 bits 12345678 e que ele seja armazenado em uma palavra de 32 bits na memória endereçável por byte no local de byte 184. O valor consiste em 4 bytes, com o byte menos significativo contendo o valor 78 e o byte mais significativo contendo o valor 12. Existem duas maneiras óbvias de armazenar esse valor:

Endereço	Valor	Endereço	Valor
184	12	184	78
185	34	185	56
186	56	186	34
187	78	187	12

O mapeamento à esquerda armazena o byte mais significativo no endereço de byte numérico mais baixo; isso é conhecido como **big-endian**, e é equivalente à ordem da esquerda para a direita da escrita nas linguagens de cultura ocidental. O mapeamento à direita armazena o byte menos significativo no endereço de byte numérico mais baixo; isso é conhecido como **little-endian**, e é um remanescente da ordem da direita para a esquerda das operações aritméticas em unidades aritméticas.⁴ Para determinado valor escalar em múltiplos bytes, *big-endian* e *little-endian* são mapeamentos com inversão de bytes um em relação ao outro.

O conceito de tipo de *endian* surge quando é necessário tratar uma entidade de múltiplos bytes como um único item de dados com um único endereço, embora seja composto de unidades endereçáveis menores. Algumas máquinas, como Intel 80x86, x86, VAX e Alpha, são máquinas *little-endian*, enquanto outras, como IBM System 370/390, Motorola 680x0, Sun SPARC e a maioria das máquinas RISC, são *big-endian*. Isso apresenta problemas quando os dados são transferidos de uma máquina de um tipo de *endian* para a outra e quando um programador tenta manipular bytes ou bits individuais dentro de um escalar de múltiplos bytes.

⁴ Os termos *big endian* e *little endian* vêm da Parte I, Capítulo 4 de *As Viagens de Gulliver*, de Jonathan Swift. Eles se referem a uma guerra religiosa entre dois grupos, um que quebra ovos na ponta grande (*big end*) e o outro que quebra ovos na ponta pequena (*little end*).

A propriedade de tipo de *endian* não se estende além de uma unidade de dados individual. Em qualquer máquina, agregados como arquivos, estruturas de dados e *arrays* são compostas de múltiplas unidades de dados, cada uma com um tipo de *endian*. Assim, a conversão de um bloco de memória de um estilo de tipo de *endian* para outro requer conhecimento da estrutura de dados.

A Figura 10.18 ilustra como o tipo de *endian* determina o endereçamento e a ordem de byte. A estrutura em C no topo contém uma série de tipos de dados. O layout da memória no canto inferior esquerdo resulta da compilação da estrutura para uma máquina *big-endian*, e a do canto inferior direito, daquela para uma máquina *little-endian*. De qualquer forma, a memória é representada como uma série de linhas de 64 bits. Para o caso do *big-endian*, a memória normalmente é vista da esquerda para a direita, de cima para baixo, enquanto para o caso *little-endian*, a memória normalmente é vista como da direita para a esquerda, de cima para baixo. Observe que esses layouts são arbitrários. Qualquer esquema poderia ser da esquerda para a direita ou da direita para a esquerda dentro de uma linha; essa é uma questão de representação, e não de atribuição de memória. De fato, examinando os manuais de programador para diversas máquinas, diversas representações podem ser encontradas, até mesmo dentro do mesmo manual.

Podemos fazer várias observações sobre essa estrutura de dados:

- Cada item de dados tem o mesmo endereço nos dois esquemas. Por exemplo, o endereço da palavra dupla com valor hexadecimal 2122232425262728 é 08.
- Dentro de determinado valor escalar multibyte, a ordenação dos bytes na estrutura *little-endian* é o reverso daquela para a estrutura *big-endian*.
- O tipo de *endian* não afeta a ordenação dos itens de dados dentro de uma estrutura. Assim, a palavra de quatro caracteres *c* exibe reversão de byte, mas o *array* de bytes de sete caracteres *d* não. Logo, o endereço de cada elemento individual de *d* é o mesmo nas duas estruturas.

O efeito do tipo de *endian* talvez seja demonstrado mais claramente quando vemos a memória como um *array* vertical de bytes, como mostra a Figura 10.19.

Não existe um consenso geral sobre qual é o estilo superior de tipo de *endian*.⁵ Os pontos a seguir favorecem o estilo *big-endian*:

- **Classificação de *string* de caracteres:** um processador *big-endian* é mais rápido em comparação com *strings* de caracteres alinhadas por inteiros; a ALU de inteiros pode comparar múltiplos bytes em paralelo.
- **Listagem de valores decimais IRA:** todos os valores podem ser impressos da esquerda para a direita sem causar confusão.

Figura 10.18 Exemplo de estrutura de dados em C e seus mapas de *endian*

		<pre> struct{ int a; //0x1112_1314 int pad; // double b; //0x2122_2324_2526_2728 char* c; //0x3132_3334 char d[7]; //'A','B','C','D','E','F','G' short e; //0x5152 int f; //0x6162_6364 } s; </pre>																word
																		doubleword
																		word
																		byte array
																		halfword
																		word

Endereço de byte	Mapeamento de endereço big-endian																	Endereço de byte
	11	12	13	14	04	05	06	07	21	22	23	24	25	26	27	28		
00	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	00	
08	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13	14	15	16	17	08	
10	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	10	
18	18	19	1A	1B	1C	1D	1E	1F	20	21	22	23	24	25	26	27	18	
20	20	21	22	23	24	25	26	27	28	29	2A	2B	2C	2D	2E	2F	20	

Endereço de byte	Mapeamento de endereço little-endian																	Endereço de byte
	07	06	05	04	03	02	01	00	0F	0E	0D	0C	0B	0A	09	08		
00	07	06	05	04	03	02	01	00	0F	0E	0D	0C	0B	0A	09	08	00	
08	0F	0E	0D	0C	0B	0A	09	08	17	16	15	14	13	12	11	10	08	
10	17	16	15	14	13	12	11	10	1F	1E	1D	1C	1B	1A	19	18	10	
18	1F	1E	1D	1C	1B	1A	19	18	27	26	25	24	23	22	21	20	18	
20	27	26	25	24	23	22	21	20	2F	2E	2D	2C	2B	2A	29	28	20	

⁵ O profeta reverenciado pelos dois grupos nas Endian Wars de *Viagens de Gulliver* disse isto: "Todos os que me acreditam verdadeiramente deverão quebrar seus ovos pela ponta mais conveniente". Isso não ajuda muito!

Figura 10.19 Outra visão da Figura 10.18

00	11	00	14
	12		13
	13		12
	14		11
04		04	
08	21	08	28
	22		27
	23		26
	24		25
0C	25	0C	24
	26		23
	27		22
	28		21
10	31	10	34
	32		33
	33		32
	34		31
14	'A'	14	'A'
	'B'		'B'
	'C'		'C'
	'D'		'D'
18	'E'	18	'E'
	'F'		'F'
	'G'		'G'
1C	51	1C	52
	52		51
20	61	20	64
	62		63
	63		62
	64		61

(a) *Big-endian*(b) *Little-endian*

- **Ordem coerente:** processadores *big-endian* armazenam seus inteiros e *strings* de caracteres na mesma ordem (byte mais significativo vem primeiro).

Os seguintes pontos favorecem o estilo *little-endian*:

- O processador *big-endian* precisa realizar adição quando converte um endereço de inteiros de 32 bits para um endereço de inteiros de 16 bits, para usar os bytes menos significativos.
- É mais fácil realizar a aritmética de alta precisão com o estilo *little-endian*; você não precisa encontrar o byte menos significativo e recuar.

As diferenças são menores e a escolha do estilo de *endian* normalmente é mais uma questão de acomodar as máquinas anteriores do que qualquer outra coisa.

O PowerPC é um processador *bi-endian*, que tem suporte para os modos *big-endian* e *little-endian*. A arquitetura *bi-endian* permite que os desenvolvedores de software escolham qualquer um desses modos ao migrar sistemas operacionais e aplicações de outras máquinas. O sistema operacional estabelece o modo de *endian* em que os processos são executados. Quando um modo é selecionado, todos os loads e stores de memória subsequentes são determinados pelo modelo de endereçamento de memória desse modo. Para dar suporte a esse recurso do hardware, 2 bits são mantidos no registrador de estado de máquina (*MSR*, do inglês *machine state register*) mantido pelo sistema operacional como parte do estado do processo. Um bit especifica o modo de *endian* em que o kernel executa; o outro especifica o modo operacional atual do processador. Assim, o modo pode ser alterado com base em cada processo.

Ordenação de bit

Na ordenação dos bits dentro de um byte, encaramos imediatamente duas questões:

1. Você conta o primeiro bit como bit zero ou como bit um?
2. Você atribui o número de bit mais baixo ao bit menos significativo do byte (*little-endian*) ou ao bit mais significativo do byte (*big-endian*)?

Essas questões não são respondidas da mesma maneira em todas as máquinas. Na realidade, em algumas máquinas, as respostas são diferentes em diferentes circunstâncias. Além do mais, a escolha da ordenação de bits *big-endian* ou *little-endian* dentro de um byte nem sempre é coerente com a ordenação *big* ou *little-endian* dentro de um escalar de múltiplos bytes. O programador precisa se preocupar com essas questões ao manipular bits individuais.

Outra área de interesse é quando os dados são transmitidos por uma linha serial de bits. Quando um byte individual é transmitido, o sistema transmite primeiro o bit mais significativo ou o bit menos significativo? O projetista precisa garantir que os bits que chegam sejam tratados corretamente. Para obter uma discussão dessa questão, veja James (1983').

Referências

- a HAYES, J. *Computer architecture and organization*. Nova York: McGraw-Hill, 1998.
- b CARTER, P. *PC Assembly Language*. Jul. 2006. Disponível no site deste livro.
- c ATKINS, M. "PC software performance tuning". *IEEE Computer*, ago. 1996.
- d PELEG, A.; WILKIE, S. e WEISER, U. "Intel MMX for multimedia PCs". *Communications of the ACM*, jan. 1997.
- e Intel Corp. *Pentium Pro and Pentium II processors and related products*. Aurora, CO. 1998.
- f BREY, B. *The Intel microprocessors: 8086/8066, 80186/80188, 80286, 80386, 80486, Pentium, Pentium Pro Processor, Pentium II, Pentium III, Pentium 4 and Core2 with 64-bit extensions*. Upper Saddle River, NJ: Prentice Hall, 2009.
- g SLOSS, A.; Symes, D. e Wright, C. *ARM system developer's guide*. San Francisco: Morgan Kaufmann, 2004.
- h KNAGGS, P. e Welsh, S. *ARM: assembly language programming*. Bournemouth University, School of Design, Engineering, and Computing, ago 2004. www.freetchbooks.com/arm-assembly-language-programming-t729.html
- i Intel Corp. *Endianness white paper*. Nov. 2004.
- j DIJKSTRA, E. "Making an ALGOL translator for the X1". In *Annual Review of Automatic Programming, Volume 4*. Pergamon, 1963.
- k COHEN, D. "On holy wars and a plea for peace". *Computer*, out. 1981.
- l JAMES, D. "Multiplexed buses: the endian wars continue". *IEEE Micro*, set. 1983.